README and notes about the template for programmers – Biber and BibLaTeX

Gerald Q. Maguire Jr.

October 2026

This document is a work in progress.

Abstract

This document describes how to use Biber data annotations in BibLaTeX entries, how to use this annotation for Contribution Roles Taxonomy (CReDiT) data, and gives some recommendations for generating fake BibLaTeX entries.

ƨ This is a very deep dive that is probably only of interest to a *very* small number of readers.

Contents

1 Roadmap: How to Read This Guide **1**

2 Using Biber data annotations in BibLaTeX entries **2**
2.1 BibLaTeX Part Annotation Signature 3
2.2 Method A for inserting ORCID links 5
2.3 Method B for inserting ORCID links 7
2.4 Using annotation to store Contributor Roles Taxonomy (CReDiT) data 9
2.5 Architectural Takeaways for Your Pipeline 14
2.6 Listing roles grouped by degree 18
2.7 Using biber data annotations for other purposes 21
2.8 Using annotations for math in titles 26
2.8.1 Can annotations be used in conjunction with reference managers 28
2.8.2 Practical Recommendation 29
2.8.3 A Forward-Looking Architecture: Split-File Overlay . . . 30
2.9 Contrasting annotations with current workflows 33
2.10 Support for Mega-Collaborations 35
2.11 Integrating annotations with current workflows 36
2.12 Scaling with the number of publications a student has in DiVA . . 40
2.12.1 Summary of Included Publications in Compilation theses (2025 Cohort) 42
2.12.2 Key Analytical Takeaways With Respect to Included Publications 43

3 Recommendations for generating fake BibLaTeX entries **44**
3.1 Authors, Titles, and other Metadata Fields 45

4 Overleaf Automation **46**
4.1 Automating the Placeholder check 46
4.2 Further Automation 70
4.3 Risks of the Method 1 based automation approach 81

References **85**

This document describes how to use Biber data annotations in BibLaTeX entries (Section 2), how to use this annotation for Contribution Roles Taxonomy (CReDiT) data (Section 2.4), and gives some recommendations for generating fake BibLaTeX entries (Section 3).

1 Roadmap: How to Read This Guide

Depending on your immediate goals with the KTH Royal Institute of Technology (KTH) thesis template, you can navigate this document using the following tracks:

Track 1 Quick Implementation (Just the ORCID & Icon Snippets)

- Go directly to Section 2
- Review Listing 1 for the ‘“author+an” syntax, and jump to Section 2.2 or Section 2.3 for the ready-to-use Lua and LaTeX hooks that automatically render clickable green ORCID badges.

Track 2 Automated Workflows & the CReDiT Python Pipeline

- Focus on Section 2.4
- Review Pattern A (Listing 5) for granular, degree-based contribution mapping.
- Skip to Section 2.11 to understand how external tools like “`CReDiT_Matrix_Wizard.py`” and “`publications_map.json`” synchronize with GitHub Actions to build contribution matrices automatically.

Track 3 Advanced Architecture & Metadata Overlays

- Read Section 2.8.1 to understand why standard reference managers (such as Zotero or JabRef) strip data annotations and how the **Split-File Overlay** “`references.bib`” + “`references-annotations.bib`” solves this.
- Explore Section 2.7 for advanced applications such as Open Access compliance tracking, math-in-title normalization, and handling mega-collaborations.

2 Using Biber data annotations in BibLaTeX entries

Biber's data annotation syntax enables bibliography entries to carry rich, structured metadata attached directly to individual author or field items without modifying the underlying BibLaTeX data model. Listing 1 shows an example with three authors, each of whom has an Open Researcher and Contributor ID (ORCID) identifier, *i.e.,* an ORCID iD. Additionally, the first two authors are noted as having contributed equally (*e.g.,* to designate joint first authorship).

As described in §3.7 of the `biblatex` package manual, data annotations enable an author to attach semantic information to information in a bibliography. The field name suffix "+an" is a user-definable marker that this field contains an annotation. After this marker is an optional named marker (by default "default"). For example, in Listing 1 one can indicate the Swedish title for a work by adding: `title+an:swedish` and the Swedish title.

Listing 1: Biber author annotations for ORCID and equal contribution

```
@article{doe2025collaborative,
  author     = {Maguire, Jr., Gerald Q. and Smith, Jane and
   ↪ Green, Alex},
  author+an = {
    1:orcid="0000-0002-6066-7469";
    1:equal="true";
    2:orcid="0000-0001-2345-6789";
    2:equal="true";
    3:orcid="0000-0003-9876-5432";
  },
  title      = {Collaborative Systems in Higher Education},
  title+an:swedish = {Samarbetssystem inom högre utbildning
   ↪ },
  journal    = {Journal of Academic Stewardship},
  year       = {2025},
  volume     = {3},
  number     = {2},
  pages      = {112--128},
  doi        = {10.1000/182},
  issn        = {0000-0000},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
```

```
}
```

Note that each line of the author annotation (shown in Listing 1) has a prefix of the form '<number>:' followed by a name, an equal sign ('='), and a quoted value. Internally, this information is encoded into *parts* and stored by `biber` in the `'.bbl'` file. Note also the use of the `placeholder` field to explicitly identify this as a fake reference to be used simply for the purposes of examples.

2.1 BibLaTeX Part Annotation Signature

Programmatic access to the annotation is done via:

```
\texttt{\textbackslash haspartannotation[<field>][<name>][<item>]\{<part>\}\{\do

\texttt{\textbackslash getpartannotation[<field>][<name>][<item>]\{<part>\}}
```

The three optional brackets define the bibliographic coordinates ('[author][default][<idx>]'), while the part slug is passed as the mandatory brace argument ('{<part>}'). In the example above, the slug will be "orcid" or "equal".

To render these annotations in print, one can define BibLaTeX formatting hooks (as shown in Listing 3 on page 5). You can test for active annotations using `\haspartannotation` and retrieve values with `\getpartannotation`. In this code, equal authorship is indicated with a superscript asterisk, while the name gets an `\orcidlink` hyperlink, *i.e.,* beside the name is the official green, circular ORCID icon as an actionable, live hyperlink targeting the author's ORCID record page. The icon itself is drawn as a resolution-independent vector path using TikZ/PDF literals rather than a raster image.

Listing 2 shows how you can generate the green, circular ORCID icon (requires use of the `orcidlink` package) or a tiny bold font green string saying "iD". Note that `\orcidlink{}` provides its own spacing after the name and before the icon.

Listing 2: ORCID icon or colored text

```
\usepackage{orcidlink}

\newcommand*{\renderorcidbadge}[1]{%
\orcidlink {#1}%
}

\newcommand*{\renderorcidbadgeText}[1]{%
\,\ href {https :// orcid . org/#1}{\ textcolor [HTML]{A6CE39}{\tiny\textbf{iD}}}%
}
```

Section 2.2 and Section 2.3 demonstrate two different methods of getting an author's ORCID iD and displaying the badge with a hyperlink to the author's ORCID landing page. First, we shall note some common features of each method: both create a TEX box (called `\orcidbox`) to render text into, both feature a new command to render the ORCID badge, each defines their own `render_orcid_from_box()` Lua function, and within a group (*i*) each defines a local `\mkbibnamefamily`* that is used when rendering a family name in a bibliography, (*ii*) sets the values of the `maxnames` and `minnames` counters to be used for the next bibliographic citation (via `\AtNextCite`), and (*iii*) uses `\fullcite{}` to render a given bibliographic entry.

The two methods differ in their definition of the `render_orcid_from_box()` or `render_orcid_clean()` Lua function and their own `\renderorcidbadgeX` function. The main difference is how they walk the contents, *i.e.,* the rendered nodes in the box. Some of the questions that might come to mind are: Why do we need to render to a box? and Why do we need to walk the contents of this box? The answer to both questions is that the `\getpartannotation` does not return a character string, but `\href` or `\orcidlink` operate in URL-string mode; hence, it requires an evaluated, literal string. By setting the result of `\getpartannotation` in a box, we can now examine its contents and turn it into a string. The code also takes advantage of the fixed format of an ORCID iD to avoid problems with hyphenation and discretionary hyphenation (this is a deep morass in the LATEX engine used, the font used, many different types of hyphens, and even ligature processing). The two methods utilize different methods of walking the nodes of the box's contents:

- Method A (Deep Recursive Walk): Walks into the sub-lists of discretionary nodes (`n.replace` and `n.pre`) directly, attempting to extract the literal glyphs embedded inside hyphenation alternatives.

- Method B (Box Descent with Direct Hyphen Substitution): Explicitly descends into nested horizontal and vertical box lists (HLIST and VLIST), treats any encountered discretionary node directly as a hyphen (`table.insert(chars, "-")`), and relies on the subsequent 16-character normalization pass to format the canonical 4-4-4-4 structure regardless of font shaping effects.

*One can also renew the command `\mkbibnamegiven` for markup of the author's first name. Similarly, the other parts of a name can be utilized: `\mkbibnameprefix` and `\mkbibnamesuffix`.

2.2 Method A for inserting ORCID links

As an example of this for \fullcite{doe2025collaborative} using the code in Listing 3 is:

[Unresolved Placeholder: Placeholder fake reference for use in examples]Gerald Q. Maguire Jr.*, Jane Smith*, and Alex Green, "Collaborative systems in higher education," *Journal of Academic Stewardship* vol. 3(2), pp. 112–128, 2025, ISSN: 0000-0000. DOI: 10.1000/182

Note that this reference has a red warning "[Unresolved Placeholder: Placeholder fake reference for use in examples]" to explicitly indicate that it is *not* a real reference. This markup is done via the field declaration in `ext-eprint-dbx.tex` and realized by the check `\iffieldundef{placeholder}` invoked by `\AtEveryCitekey` (executes when a citation key is processed in text, *e.g.,* `\cite`, `\parencite`, `\textcite`, and `\fullcite`) and `\AtEveryBibitem` (executes inside the bibliography list, *i.e.,* `\printbibliography`). Because `\fullcite` internally expands using bibliography drivers, there is no double-printing or omission of the warning on standalone citations in the log file.

Listing 3: Example of a LaTeX hook for equal contribution and ORCID icons

```
\newbox\orcidbox

\newcommand*{\renderorcidbadge}[1]{%
\orcidlink {#1}%
}

\begin{luacode*}
function render_orcid_from_box()
  local box_head = tex.box.orcidbox.head
  if not box_head then return end

  local chars = {}
  local GLYPH = node.id("glyph")
  local DISC = node.id("discretionary")

  local function walk(head)
    for n in node.traverse(head) do
      if n.id == GLYPH then
        table.insert(chars, utf8.char(n.char))
      elseif n.id == DISC then
        if n.replace then
          walk(n.replace)
        elseif n.pre then
```

```
            walk(n.pre)
          else
            table.insert(chars, "-")
          end
        elseif n.head then
          walk(n.head)
        end
      end
    end

    walk(box_head)
    local raw_orcid = table.concat(chars)

    -- Filter out anything except digits and X/x (hyphen placed at the very end of class
    ↪ )
    local base = ""
    for c in raw_orcid:gmatch("[0-9xX]") do
      base = base .. c
    end

    local formatted_orcid = ""
    if #base == 16 then
      -- Normalize the 16th character (check digit) to uppercase X if applicable
      local prefix = base:sub(1, 15)
      local checksum = base:sub(16, 16):upper()
      local full = prefix .. checksum

      formatted_orcid = string.format("%s-%s-%s-%s",
        full:sub(1, 4),
        full:sub(5, 8),
        full:sub(9, 12),
        full:sub(13, 16)
      )
    else
      formatted_orcid = raw_orcid
    end

    if formatted_orcid and formatted_orcid ~= "" then
      tex.sprint(token.create("renderorcidbadge"), "{", formatted_orcid, "}")
    end
  end
\end{luacode*}

\begingroup
\renewcommand*{\mkbibnamefamily}[1]{%
```

```
  #1%
  \edef\curridx {\the\value{listcount }}%
  \haspartannotation [author][ default ][\ curridx ]{ equal }{%
    \textsuperscript {*}%
  }{}%
  \haspartannotation [author][ default ][\ curridx ]{ orcid }{%
    \global\setbox\orcidbox=\hbox{\getpartannotation [author][ default ][\ curridx ]{ orcid
    ↪ }}%
    \directlua {render_orcid_from_box()}%
  }{}%
}

\AtNextCite{%
  \defcounter {maxnames}{99}%
  \defcounter {minnames}{99}%
}

\fullcite {doe2025collaborative }
\endgroup
```

2.3 Method B for inserting ORCID links

Another example of inserting ORCID links for \fullcite{doe2025collaborative} using the code in Listing 4 is:

[Unresolved Placeholder: Placeholder fake reference for use in examples]Gerald Q. Maguire Jr.*, Jane Smith*, and Alex Green, "Collaborative systems in higher education," *Journal of Academic Stewardship* vol. 3(2), pp. 112–128, 2025, ISSN: 0000-0000. DOI: 10.1000/182

Listing 4: Example of a LaTeX hook for equal contribution and ORCID icons

```
\usepackage{biblatex}
\usepackage{orcidlink}

\newbox\orcidbox

\newcommand*{\renderorcidbadgeB}[1]{%
\orcidlink {#1}%
}

\begin{luacode*}
function render_orcid_clean()
  local box_head = tex.box.orcidbox.head
  if not box_head then return end
```

```
local chars = {}
local GLYPH = node.id("glyph")
local DISC = node.id("discretionary")
local HLIST = node.id("hlist")
local VLIST = node.id("vlist")

local function walk(head)
  for n in node.traverse(head) do
    if n.id == GLYPH then
      table.insert(chars, utf8.char(n.char))
    elseif n.id == DISC then
      table.insert(chars, "-")
    elseif n.id == HLIST or n.id == VLIST then
      if n.head then walk(n.head) end
    end
  end
end

walk(box_head)
local raw_orcid = table.concat(chars)

-- 1. Extract only alphanumeric characters (digits and X/x)
local alphanum = ""
for c in raw_orcid:gmatch("[0-9xX]") do
  alphanum = alphanum .. c
end

-- 2. Format into standard ORCID 4-4-4-4 hyphenated layout
local final_orcid = ""
if #alphanum == 16 then
  local prefix = alphanum:sub(1, 15)
  local checksum = alphanum:sub(16, 16):upper()
  local full = prefix .. checksum

  final_orcid = string.format("%s-%s-%s-%s",
    full:sub(1, 4),
    full:sub(5, 8),
    full:sub(9, 12),
    full:sub(13, 16)
  )
else
  final_orcid = raw_orcid
end
```

```
  if final _orcid and final _orcid ~= "" then
    tex.sprint(token.create("renderorcidbadgeB"), "{", final _orcid, "}")
  end
end
\end{luacode*}

\begingroup
\renewcommand*{\mkbibnamefamily}[1]{%
  #1%
  \edef\curridx {\the\value{listcount }}%
  \haspartannotation [author][ default ][\ curridx ]{ equal }{%
    \textsuperscript {*}%
  }{}%
  \haspartannotation [author][ default ][\ curridx ]{ orcid }{%
    \global\setbox\orcidbox=\hbox{\getpartannotation [author][ default ][\ curridx ]{ orcid
    ↪ }}%
    \directlua {render_orcid_clean() }%
  }{}%
}

\AtNextCite{%
  \defcounter {maxnames}{99}%
  \defcounter {minnames}{99}%
}

\fullcite {doe2025collaborative }
\endgroup
```

2.4 Using annotation to store Contributor Roles Taxonomy (CReDiT) data

Currently, there are no hardcoded built-in keyword annotations reserved specifically for CReDiT (Contributor Roles Taxonomy, ANSI/NISO Z39.104-2022 [2]). However, standard canonical identifiers using the "Simplified Name" form can be utilized directly as annotation keys, as outlined in Table 1. The degree values of these can be: "lead", "equal" (=true), or "supporting":

lead The contributor took primary responsibility for the facet of the work.

equal The contributor shared substantial, balanced responsibility alongside one or more co-authors.

supporting The contributor provided supplementary contribution, operational assistance, or targeted input.

Absence of the key implies no attributed contribution.

Table 1: CReDiT roles for author data annotations in BibLaTeXwith their Simplified Name, *i.e.,* annotation key

CReDiT Role	Simplified Name
Conceptualization	`conceptualization`
Data curation	`data-curation`
Formal analysis	`formal-analysis`
Funding acquisition	`funding-acquisition`
Investigation	`investigation`
Methodology	`methodology`
Project administration	`project-administration`
Resources	`resources`
Software	`software`
Supervision	`supervision`
Validation	`validation`
Visualization	`visualization`
Writing – original draft	`writing-original-draft`
Writing – review & editing	`writing-review-editing`

There are two primary paradigms for encoding CReDiT roles using `biber` annotations, as shown in Listing 5 and Listing 6.

Listing 5: Pattern A: Role-as-Key (Fine-grained with degrees of contribution)

```
@article{maguire2026pipeline,
  author    = {Maguire Jr., Gerald Q. and Smith, Jane},
  author+an = {
    1:orcid="0000-0002-6066-746X";
    1:conceptualization="lead";
    1:software="lead";
    1:writing-original-draft="lead";
    2:orcid="0000-0001-2345-6789";
    2:supervision="lead";
    2:writing-review-editing="supporting"
  },
```

```
  title     = {Automated Metadata Integrity in Academic
   ↪ Publishing},
  journal   = {Journal of Academic Stewardship},
  year      = {2026},
  volume    = {4},
  number    = {1},
  pages     = {45--60},
  doi       = {10.1000/182},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}
```

Listing 6: Pattern B: Single credit container (Comma-separated token string)

```
@article{maguire2026pipeline,
  author    = {Maguire, Jr., Gerald Q. and Smith, Jane},
  author+an = {
    1:orcid="0000-0002-6066-746X"; 1:credit="
   ↪ Conceptualization, Software, Writing - original draft"
   ↪ ;
    2:orcid="0000-0001-2345-6789"; 2:credit="Supervision,
   ↪ Writing - review & editing"
  },
  title     = {Automated Metadata Integrity in Academic
   ↪ Publishing},
  journal   = {Journal of Academic Stewardship},
  year      = {2026},
  placeholder = {A fake example for documentation}
}
```

Pattern A is the best method for KTH doctoral thesis students because it makes it easy to state the values associated with a role.

Listing 7: Example LaTeX hook for extracting and rendering contributor roles

```
\usepackage{biblatex}

% Macro to iterate through author list and render contributor role statements
\newbibmacro*{author:credit:roles}{%
  \begingroup
  \def\do{%
    \haspartannotation[author][default][\currentname]{credit}%
      {\par\small\textbf{\namepartgiven} \textbf{\namepartfamily}: \getpartannotation[
      ↪ author][default][\currentname]{credit}}%
      {}%
  }%
  \printnames[creditnames]{author}%
  \endgroup
}

% Append contribution details to extended bibliography listings
\renewbibmacro*{finentry}{%
  \usebibmacro{author:credit:roles}%
  \finentry %
}
```

This setup allows automated Python/Continuous Integration (CI) validation pipelines to extract and cross-check contributor statements directly from `.bib` files without manual re-entry.

Because `\renewbibmacro*` is dynamically scoped in LaTeX, it can be redefined specifically for the author contribution section and either restored afterward or restricted within a local group.

As shown in Listing 8, wrapping the contribution section inside a TeX group (`\begingroup …\endgroup`) ensures the modification automatically discards itself when the scope closes, leaving the main thesis bibliography unaffected:

Listing 8: Local group scoping for CReDiT bibliography output

```
\chapter*{Author Contributions }

\begingroup
% 1. Redefine finentry locally to append CReDiT statements
\renewbibmacro*{finentry}{%
  \usebibmacro{author:credit :roles }%
  \finentry %
}

\textbf {Paper I }
\fullcite {maguire2026pipeline}

\textbf {Paper II }
\fullcite {doe2025collaborative }
\endgroup

% Outside the group, finentry reverts to its original standard definition
```

The following generates some example output when using Approach A for the CReDiT information in the `.bib` file:

Paper I

[Unresolved Placeholder: Placeholder fake reference for use in examples]Gerald Q. Maguire Jr. and Jane Smith, "Automated metadata integrity in academic publishing," *Journal of Academic Stewardship* vol. 4(1), pp. 45–60, 2026, ISSN: 0000-0000. DOI: 10.1000/182

Author Contributions:

Gerald Q. Maguire Jr.

Conceptualization: *lead*

Software: *lead*

Writing – original draft: *lead*

Jane Smith

Supervision: *lead*

Writing – review & editing: *supporting*

Paper II

[Unresolved Placeholder: Placeholder fake reference for use in examples]Gerald Q. Maguire Jr. et al., "Collaborative systems in higher education," *Journal of Academic Stewardship* vol. 3(2), pp. 112–128, 2025, ISSN: 0000-0000. DOI: 10.1000/182

Author Contributions:

Gerald Q. Maguire Jr.
Investigation: *lead*
Writing – original draft: *equal*
Equal contribution: *true*

Jane Smith
Data curation: *lead*
Writing – original draft: *equal*
Equal contribution: *true*

Alex Green
Supervision: *lead*
Writing – review & editing: *supporting*

2.5 Architectural Takeaways for Your Pipeline

- **biber ".bib" Annotation Delimiters:** Item annotations must be delimited by semicolons (';'), not commas (','), with the item index explicit on each part (*e.g.,* '1:conceptualization="lead"; 1:software="lead"').
- **Kebab-Case Slugs:** Adopting the standard CReDiT simplified names (*e.g.,* 'writing-original-draft', 'data-curation') avoids unescaped underscore collisions in TeX math mode and simplifies downstream JavaScript Object Notation (JSON) serialization.
- **BibLaTeX Part Annotation Signature:**

```
\texttt{\textbackslash haspartannotation[<field>][<name>][<item>]\{<p
```

```
\texttt{\textbackslash getpartannotation[<field>][<name>][<item>]\{<p
```

The three optional brackets define the bibliographic coordinates ('[author][default][<idx>]'), while the part slug is passed as the mandatory brace argument ('{<part>}').

- **Index Isolation:** Freezing ‘\the\valuelistcount‘ via ‘\edef‘ before triggering ‘\docsvlist‘ prevents the inner loop from clobbering the active author index.

If you prefer to redefine macros globally across document sections, redefine `finentry` before the papers list and reset it before calling `\printbibliography`:

Listing 9: Sequential document-level redefinition of finentry

```
% In the Author Contributions front matter:
\renewbibmacro*{finentry}{%
  \usebibmacro{author:credit:roles}%
  \finentry %
}

\fullcite {maguire2026pipeline}

% ------------------------------------------------------------
% Later, in the back matter before the main thesis bibliography:
% ------------------------------------------------------------

\renewbibmacro*{finentry}{\finentry }

\printbibliography [heading=bibintoc, title ={References}]
```

For granular control when citing individual papers with contributor details without modifying global hooks, encapsulate the scoped redefinition inside a custom macro:

Listing 10: Defining a dedicated \fullcitecredit command

```
% Dedicated command encapsulating scoped CReDiT redefinition
\newcommand{\fullcitecredit}[1]{%
  \begingroup
  \renewbibmacro*{finentry}{%
    \usebibmacro{author:credit:roles}%
    \finentry %
  }%
  \fullcite {#1}%
  \endgroup
}
```

BibLaTeX provides the author index context automatically via `\currentname` (which expands to the current name list position, such as 1, 2, *etc.*) when stepping through an author list with `\indexnames` or custom name formatters.

Define the slug-to-text mappings with `\csdef`, as shown in Listing 11. Using `etoolbox` list iteration (`\docsvlist`), you can step through each author and

test each of the 14 standard CReDiT slugs systematically. Note that the code in Listing 11 should be placed in the preamble, except for "6. Hook into finentry" which only needs to be invoked (within a `\begingroup` … `\endgroup`) when you need it for listing the author contributions in your thesis.

Listing 11: Iterate through slugs

```
\usepackage{biblatex}
\usepackage{etoolbox}

% 1. Helper mapping from simplified kebab-case slug to display text
\newcommand{\formatcreditrole}[1]{%
  \ifcsdef {credit@#1}{\csuse{credit@#1}}{\detokenize{#1}}%
}
\csdef{credit@conceptualization }{Conceptualization }
\csdef{credit@data-curation }{Data curation }
\csdef{credit@formal-analysis }{Formal analysis }
\csdef{credit@funding-acquisition }{Funding acquisition }
\csdef{credit@investigation }{Investigation }
\csdef{credit@methodology}{Methodology}
\csdef{credit@project -administration }{Project administration }
\csdef{credit@resources }{Resources}
\csdef{credit@software}{Software}
\csdef{credit@supervision }{Supervision}
\csdef{credit@validation }{Validation }
\csdef{credit@visualization }{Visualization }
\csdef{credit@writing-original -draft }{Writing -- original draft }
\csdef{credit@writing-review-editing }{Writing -- review \& editing }
\csdef{credit@equal}{Equal contribution }

% 2. List of CReDiT simplified slugs to check
\newcommand*{\CreditSlugList}{%
  conceptualization , data-curation , formal-analysis , funding-acquisition , %
  investigation , methodology,project -administration , resources , software , %
  supervision , validation , visualization , writing -original -draft , writing -review-editing ,
    ↪ equal%
}

% 3. Check and print role :
\newcommand*{\checkandprintrole}[2]{%
  \haspartannotation [author][ default ][#1]{#2}{%
    \par\hspace*{1.5em}\formatcreditrole{#2}: \textit {\getpartannotation [author][
    ↪ default ][#1]{#2}}%
  }{}%
}
```

```
% 4. Name format driver
\DeclareNameFormat{creditnames}{%
  \edef\currentauthornum{\the\value{listcount }}%
  \par\smallskip\noindent\textbf {\namepartgiven} \textbf {\namepartfamily}%
  \def\do##1{\checkandprintrole {\currentauthornum}{##1}}%
  \expandafter\docsvlist \expandafter{\CreditSlugList }%
}

% 5. Bibmacro: forces evaluation of all authors
\newbibmacro*{author:credit:roles }{%
  \begingroup
  \defcounter {maxnames}{99}%
  \defcounter {minnames}{99}%
  \par\medskip\noindent\textbf{Author Contributions :} %
  \printnames[creditnames]{author }%
  \endgroup
}

% 6. Hook into finentry
\renewbibmacro*{finentry}{%
  \finentry
  \usebibmacro{author:credit :roles }%
}
```

In BibLaTeX, whenever a citation or bibliography driver is executing (such as inside `\AtEveryBibitem`, `\AtEveryCitekey`, `\DeclareBibliographyDriver`, `\fullcite`, or inside a bibmacro like `finentry`), the active entry key is automatically set as the current context.

When writing custom reports, contribution matrices, or diagnostic validation routines outside the standard citation loop, BibLaTeX provides several mechanisms to inspect and manipulate the active citation context:

1. Standard Built-in Key Registers

   - `\thefield{entrykey}`: Prints or retrieves the active `.bib` entry key (*e.g.,* maguire2026pipeline).
   - `\strfield{entrykey}`: Returns the unexpanded string representation of the entry key.
   - `\currententrykey`: An internal macro holding the literal key of the item currently being processed.

2. Explicitly Setting an Arbitrary Entry as “Current”

If you are outside a standard bibliography loop (for example, in custom TeX/Lua code or arbitrary body text) and want to load a specific entry key into the active BibLaTeX evaluation context without printing a full citation:

Listing 12: Load specific entry key

```
\begingroup
% Switch the active biblatex context to 'myEntryKey'
\entryclone {myEntryKey}{current_context_key} % Optional clone
% or set the entry data context directly :
\usedatacontext [entrykey=myEntryKey]
...
\endgroup
```

More commonly, BibLaTeX provides `\entrydata{<entrykey>}{<code/macros>}`, which temporarily switches the current entry context to `<entrykey>` and executes your custom code within that entry's data scope:

Listing 13: Temporary switch to specific entry key

```
\entrydata {maguire2026pipeline}{%
  % Inside this block, 'maguire2026pipeline' is the active entry
  \textbf {Active Key:} \thefield {entrykey}\par
  \textbf {Title :} \printfield {title }\par
  \usebibmacro{author:credit :roles }%
}
```

Example usage to print the current entry key alongside metadata;

Listing 14: Print current entry key with metadata

```
\newbibmacro*{print:current :entry :info }{%
  \par\noindent
  \textbf {Entry Key:} \thefield {entrykey}\par
  \textbf {Entry Type:} \thefield {entrytype }\par
  \textbf {Active Authors:} \printnames{author}%
}
```

2.6 Listing roles grouped by degree

Whenever the `author:credit:roles` macro is invoked during bibliography rendering, `\thefield{entrykey}` dynamically evaluates to whatever reference is currently being processed. Listing 15 shows how the roles can be listed grouped by degree. If an author has no roles allocated under a specific degree (such as no Equal roles), that row is omitted entirely.

Listing 15: Listing CReDiT roles grouped by degree

```
\usepackage{etoolbox}

% Master list of the 14 standard CReDiT roles
\newcommand*{\CreditRolesList}{%
  conceptualization ,%
  data-curation ,%
  formal-analysis ,%
  funding-acquisition ,%
  investigation ,%
  methodology,%
  project -administration ,%
  resources ,%
  software ,%
  supervision ,%
  validation ,%
  visualization ,%
  writing -original -draft ,%
  writing -review-editing %
}

% Human-readable labels for role slugs
\newcommand*{\formatcreditrole}[1]{%
  \ifcsdef {creditlabel@ #1}%
    {\csuse{creditlabel@ #1}}%
    {#1}%
}
\csdef{creditlabel@conceptualization }{Conceptualization }
\csdef{creditlabel@data -curation }{Data Curation}
\csdef{creditlabel@formal -analysis }{Formal Analysis}
\csdef{creditlabel@funding -acquisition }{Funding Acquisition }
\csdef{creditlabel@investigation }{Investigation }
\csdef{creditlabel@methodology}{Methodology}
\csdef{creditlabel@project -administration }{Project Administration }
\csdef{creditlabel@resources }{Resources}
\csdef{creditlabel@software }{Software}
\csdef{creditlabel@supervision }{Supervision}
\csdef{creditlabel@validation }{Validation }
\csdef{creditlabel@visualization }{Visualization }
\csdef{creditlabel@writing -original -draft }{Writing -- Original Draft}
\csdef{creditlabel@writing -review-editing }{Writing -- Review \& Editing}

% Formatting template for each degree group
\newcommand*{\printcreditgroup}[2]{%
  \ifcsvoid {#1}{}{%
```

```
    \par\hskip 1.5em\relax
    \textit {#2:} \csvtolabel {#1}%
  }%
}

% Comma-separated output helper for role lists
\newcommand*{\csvtolabel}[1]{%
  \begingroup
  \def\do##1{\formatcreditrole {##1}\def\do{, \formatcreditrole {####1}}}%
  \dolistcsloop {#1}%
  \endgroup
}

% Grouped synthesis name-formatting driver
\DeclareNameFormat{creditgrouped}{%
  \begingroup
  % Initialize accumulators for each contribution degree
  \csgdef{credit@lead@\the\value{listcount }}{}%
  \csgdef{credit@equal@\the\value{listcount }}{}%
  \csgdef{credit@supporting@\the\value{listcount }}{}%
  \togglefalse {hascredit }%
  %
  % Iterate through all 14 roles for the current author index
  \renewcommand*{\do}[1]{%
    \haspartannotation [author][##1][\ the\value{listcount }]%
      {%
        \toggletrue {hascredit }%
        \edef\curdegree{\getpartannotation [author][##1][\ the\value{listcount }]}%
        \ifdefstring {\curdegree}{lead}%
          {\listcsgadd {credit@lead@\the\value{listcount }}{##1}}{}%
        \ifdefstring {\curdegree}{equal}%
          {\listcsgadd {credit@equal@\the\value{listcount }}{##1}}{}%
        \ifdefstring {\curdegree}{supporting }%
          {\listcsgadd {credit@supporting@\the\value{listcount }}{##1}}{}%
      }{}%
  }%
  \docsvlist {\CreditRolesList }%
  %
  % Typeset author and grouped roles only if contributions are registered
  \iftoggle {hascredit }{%
    \par\smallskip\noindent
    \textbf {%
      \ifdefvoid {\namepartgiven}{}{\namepartgiven\ }%
      \ifdefvoid {\namepartfamily}{}{\namepartfamily}%
      \ifdefvoid {\namepartjunior }{}{ \namepartjunior }%
```

```
    }%
    \haspartannotation  [author][ orcid ][\ the\value{listcount  }]%
      {\space\footnotesize(\url {https ://  orcid . org/\ getpartannotation  [author][ orcid ][\
    ↪  the\value{listcount  }]}) }%
      {}%
    \par\nopagebreak
    \printcreditgroup  {credit@lead@\the\value{listcount  }}{Lead}%
    \printcreditgroup  {credit@equal@\the\value{listcount  }}{Equal}%
    \printcreditgroup  {credit@supporting@\the\value{listcount  }}{Supporting}%
  }{}%
  \endgroup
}

% Replacement master macro
\newbibmacro*{author:credit : roles  }{%
  \begingroup
  \newtoggle{hascredit  }%
  \printnames[creditgrouped ]{author }%
  \endgroup
}
```

Using creditlabel@… provides strict separation:

creditlabel@<role>	Static translation table mapping raw slug keys to typeset strings.
credit@<degree>@<author_idx>	Dynamic runtime data containers accumulating roles during macro evaluation.

2.7 Using biber data annotations for other purposes

Beyond author attribution and multilingual titles, Biber's data annotations can attach semantic metadata to citations, authors, and sources without requiring custom `.dbx` datamodel extensions. Potentially, several applications could be integrated directly into thesis workflows:

1. Thesis Inclusion and Paper Categorization (Compilation Theses)

   In Swedish compilation doctoral theses (sammanläggningsavhandlingar), candidates must distinguish between papers formally included in the thesis and secondary publications completed during their studies:

Listing 16: Storing thesis related metadata

```
@article{maguire2025routing,
  author     = {Maguire, Jr., Gerald Q. and Student,
   ↪ Alice},
  entry+an   = {included="true"; paper="Paper I"},
  title      = {Adaptive Mesh Architectures in 6G
   ↪ Deployments},
  journal    = {IEEE Transactions on Networking},
  year       = {2025},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}

@inproceedings{student2024early,
  author     = {Student, Alice and Maguire, Jr.,
   ↪ Gerald Q.},
  entry+an   = {included="false"; category="other"},
  title      = {Preliminary Bounds on Packet
   ↪ Dispersion},
  booktitle  = {Proceedings of ACM SIGCOMM Posters},
  year       = {2024},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}
```

   Use in Document A single `.bib` file can populate both the **Included Papers** section and the **Other Publications** appendix by filtering on `\hasentryannotation{included}` inside `\defbibfilter` or `\defbibcheck`.

   Automated Dividers The value `paper="Paper I"` can be dynamically fetched to typeset decorative Roman-numeral divider sheets preceding individual reprint Portable Document Formats (PDFs).

2. Institutional Affiliation and Degree Eligibility Mapping

   For co-authored papers, evaluating committees often need to confirm whether an author was affiliated with KTH or performed the research under an external industrial partnership:

Listing 17: Storing thesis related metadata

```
@article{smith2026telecom,
  author    = {Smith, Jane and Maguire, Jr., Gerald Q.
   ↪ and Larsson, Sven},
  author+an = {
    1:affiliation="kth"; 1:school="EECS";
    2:affiliation="kth"; 2:school="EECS";
    3:affiliation="external"; 3:org="Ericsson Research
   ↪ "
  },
  title     = {Deterministic Latency Guarantees in TSN
   ↪ },
  journal   = {Computer Networks},
  year      = {2026},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}
```

Use in Document Footnotes or superscript markers can identify authors belonging to specific departments or external bodies without cluttering standard citation styles.

MODS / DiVA Generation The Python pipeline can inspect `affiliation` and `org` to construct `<affiliation>` and corporate name nodes for Digitala Vetenskapliga Arkivet (DiVA) without manual disambiguation.

While footnotes or markers to identify authors belonging to specific departments or external bodies might be possible, I think they add too much cognitive load for the reader; therefore, it is better to include these descriptions in the text. Moreover, a better means to input metadata into DiVA is needed (potentially, through its new Application Programming Interface (API)); however, this lies outside the scope of the current work.

3. Open Access (OA) & Archival Compliance Status

Institutional repositories and funding bodies (such as the Swedish Research Council / Vetenskapsrådet) track whether listed papers meet Plan S open-access criteria:

Listing 18: Storing Open Access (OA) & Archival Compliance Status

```
@article{johnson2026protocols,
  author    = {Johnson, Mark and Maguire, Jr., Gerald Q
   ↪ .},
  entry+an  = {oa="gold"; license="cc-by-4.0";
   ↪ repository="diva:123456"},
  title     = {P4 Acceleration on SmartNIC Clusters},
  journal   = {IEEE Communications Surveys & Tutorials
   ↪ },
  year      = {2026},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}
```

Use in Document Badges (*e.g.*, small open-access locks or CC icons) can be rendered beside citations in the list of publications.

Verification Scripts A Python/CI pre-flight check can flag any included paper annotated as ‘oa=”closed”‘ or missing a repository identifier.

This looks like a clear area for future work. It can build on the earlier ORCID code.

4. Reproducibility, Artifact, and Dataset Linking

Conferences and journals increasingly evaluate research artifacts (software, raw datasets, benchmarks). Annotating field-level URLs clarifies the target resource:

Listing 19: Storing Reproducibility, Artifact, and Dataset Linking

```
@article{turner2025inference,
  author    = {Turner, Bob and Smith, Jane},
  url       = {https://doi.org/10.1000/182},
  url+an    = {type="publication"},
  eprint    = {10.5281/zenodo.1234567},
  eprint+an = {type="dataset"; badge="zenodo"},
  note+an   = {artifact="acm-reproduced-badge"},
  title     = {cuBLAS Acceleration for Structured
   ↪ Pruning},
  journal   = {ACM Transactions on Computer Systems},
  year      = {2025},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}
```

Use in Document Field annotations on ‘eprint‘ or ‘url‘ allow custom bibliography macros to display distinct badges (*e.g.,* Zenodo dataset icon, GitHub repository badge, or ACM Reproducibility Seals) directly adjacent to the reference.

This looks like a clear area for future work. It can build on the earlier ORCID code.

5. Peer-Review Status & Acceptance Tracking

   Because doctoral theses are frequently submitted while papers are in various stages of publication, annotations can dynamically distinguish formal review stages:

   Listing 20: Storing Peer-Review Status & Acceptance Tracking

```
@article{maguire2026verification,
  author    = {Maguire, Jr., Gerald Q. and Student,
   ↪ Alice},
  entry+an  = {peerreview="accepted"; date_accepted="
   ↪ 2026-08-15"; preprint="arxiv:2608.01234"},
  title     = {Formal Verification of SmartNIC
   ↪ Microcode},
  journal   = {IEEE Micro},
  year      = {2026},
  placeholder={Placeholder fake reference for use in
   ↪ examples}
}
```

   Use in Document Bibliography listings can automatically append **"(Accepted for publication, Aug 2026)"** or **"(Under peer review)"** without hardcoding temporary text into the `journal` or `year` fields.

   Automated Guardrails Build hooks can emit compiler warnings if a student attempts to build a final dissertation (`\finaltrue`) while citations contain `peerreview="submitted"`.

   Before attempting this, more thought is needed — as this would seem difficult to maintain **and** many of the manuscripts will not be accepted (except as preprints) before the thesis is finalized and printed. However, one might consider a script-based workflow that checks the relevant DiVA entry for updated information.

2.8 Using annotations for math in titles

Some bibliographic titles contain LaTeX math expressions. BibLaTeX provides built-in literal fields for sorting and indexing, specifically `sorttitle` and `indextitle`, as illustrated in Listing 21. These fields decouple sorting and indexing strings from complex typesetting commands.

Listing 21: Using sorttitle

```
@article{Noz_1975,
  author        = {Noz, Marilyn E. and Maguire, Jr.,
      ↪  Gerald Q.},
  title         = {{Calculation of $\bar{E}_{\beta}$,
      ↪  $\Gamma$ and $\Delta_{i}$ for $^{99m}$Tc}},
  sorttitle     = {Calculation of E_beta, Gamma and
      ↪  Delta_i for 99mTc},
  …
}
```

While `sorttitle` normalizes the title into plain ASCII for alphabetical ordering, external ingest pipelines (*e.g.,* automated bibliography validators, metadata scrapers, or repository systems such as SwePub and DiVA) frequently require a clean UTF-8 plain-text representation. We can use BibLaTeX data annotations to store a canonical UTF-8 version of the title, as shown in Listing 22. Note that the leading "=" (equal sign) instructs `biber` to treat the annotation value as a literal string rather than a key-value or boolean flag.

Listing 22: Using data annotations to represent math in titles

```
@article{Noz_1975,
  author        = {Noz, Marilyn E. and Maguire, Jr.,
      ↪  Gerald Q.},
  title         = {{Calculation of $\bar{E}_{\beta}$,
      ↪  $\Gamma$ and $\Delta_{i}$ for $^{99m}$Tc}},
  title+an:utf8 = {=Calculation of Ē_β, Γ and Δ_i for ⁹⁹ᵐTc},
  …
}
```

Both approaches can be combined, as shown in Listing 23. This allows `sorttitle` to govern internal `biber` sorting while `title+an:utf8` supplies external validation pipelines and metadata harvesters with an authoritative UTF-8 string.

Listing 23: Combining data annotations with sorttitle

```
@article{Noz_1975,
  author         = {Noz, Marilyn E. and Maguire, Jr.,
      ↪ Gerald Q.},
  title          = {{Calculation of $\bar{E}_{\beta}$,
      ↪ $\Gamma$ and $\Delta_{i}$ for $^{99m}$Tc}},
  sorttitle      = {Calculation of E_beta, Gamma and
      ↪ Delta_i for 99mTc},
  title+an:utf8 = {=Calculation of Ēβ, Γ and Δᵢ for ⁹⁹ᵐTc},
  …
}
```

2.8.1 Can annotations be used in conjunction with reference managers

Considering the usage "Thesis Inclusion and Paper Categorization (Compilation Theses)" above, there is a practical problem: Currently, no mainstream reference manager supports biber's data annotations as first-class, structured semantic data.

The landscape divides into tools that parse and preserve the raw fields versus tools that strip them entirely:

1. JabRef (Best in Class, but Text-Only)

   Because JabRef uses the `.bib` file directly as its internal database (without converting entries into a generic internal schema), it is the most compatible tool:

Preservation	When set to BibLaTeX mode, JabRef will not corrupt or strip 'author+an' or 'title+an' fields when saving.
UI Editing	It does not parse the sub-properties ('1:orcid="...";') into Geraphical User Interface (GUI) checkboxes, creator cards, or linked badges. The entire annotation block appears as an unformatted raw string in an auxiliary field text box.
Feature Status	Native support for `biber` data annotations has been discussed in feature issues (*e.g.,* Issue #11505), but it remains an unparsed text field.

2. Zotero & Mendeley (Destructive / Incompatible)

   Both tools convert bibliographic entries into an internal relational model that assumes a flat "author" table ("first_name", "last_name", "role"):

   Data Loss on Import — If a `.bib` file decorated with "author+an = 1:orcid="..."; " is imported, the parser discards the annotation entirely because it does not map to any creator field.

   Export Inability — Neither tool can reconstruct "<field>+an" upon export unless an external layer like Better BibTeX (BBT) parses custom dummy strings stashed in the record's "Extra" field.

3. EndNote, Citavi, & Paperpile

   Proprietary Flat Schemas — These tools lack any concept of item/part-level annotation syntax.

   Export Stripping — They adhere to standard legacy BibTeX fields (`author`, `title`, `journal`, *etc.*). Unknown fields with plus signs ("+an") are either sanitized away or converted to malformed entries during export.

2.8.2 Practical Recommendation

Because reference managers lag behind `biber`'s advanced data-model features, **managing annotations outside the GUI reference manager is the only stable design pattern**. For a KTH doctoral workflow, managing ORCIDs and CReDiT through `publications_map.json` and external Python/CI scripts remains far more robust than relying on a reference manager.

This means:

1. `references.bib` as the standard citation repository (which Zotero or JabRef can export safely), and

2. `publications_map.json` (managed via the Python/Streamlit wizard and GitHub Actions) as the pipeline that programmatically decorates

entries with "author+an" at build time, ensuring reference managers never accidentally wipe out contributor roles or ORCIDs during library re-exports.

2.8.3 A Forward-Looking Architecture: Split-File Overlay

A cleaner architectural path forward is to decouple the reference management lifecycle from semantic metadata authoring by maintaining two distinct bibliographic files sharing identical citation keys:

1. `references.bib`: The upstream, GUI-managed store (exported directly from Zotero, Mendeley, or JabRef) containing canonical bibliographic descriptors (*e.g.,* `author`, `title`, `year`, `doi`).

2. `references-annotations.bib`: A companion overlay containing sparse entries with matching citation keys that declare only the extended data annotations (*e.g.,* `author+an`, `title+an`, or `entry+an`).

Existing utilities such as `bibtool`, `bibtex-tools`, or `bibmerge` are designed to detect duplicate citation keys and discard redundant entries. In contrast, an automated Python pre-compilation step (*e.g.,* integrated into a GitHub Action or local pre-commit hook) can perform an *additive, field-level unification*. By parsing both files and merging fields that share a matching key into a unified intermediate file (*e.g.,* `references-merged.bib`), reference managers can safely overwrite `references.bib` during iterative library exports without clobbering curated author roles or ORCIDs.

Such an overlay or split-file architecture (`references.bib` alongside `references-annotations.bib`) cleanly separates concerns while providing persistent annotations. As a result, students never have to manually re-annotate files after updating their reference library.

In `biber` and BibLaTeX, data annotations are completely generic: *any* standard or custom field can take the "+an" suffix at the item, field, or entry level.

Under the split-file overlay architecture (`references.bib` + `references-annotations.bib`), an additive Python pre-processor can attach granular semantic metadata across several distinct bibliographic dimensions without reference-manager corruption. Examples of these other dimensions include:

1. Identifier Verification & Provenance (`doi+an`, `eprint+an`, `isbn+an`)

Context In automated pipelines, it is crucial to record whether a persistent identifier has been mechanically validated against Crossref, DataCite, MathSciNet, LIBRIS, or another source.

Listing 24: Syntax for Identifier Verification & Provenance

Syntax

```
doi      = {10.1109/TNET.2025.1234567},
doi+an   = {status="verified"; resolver="
   ↪ crossref"; checked="2026-03-01"},
eprint   = {2501.09876},
eprint+an = {archive="arxiv"; primaryClass="
   ↪ cs.NI"; openaccess="true"}
```

Application BibLaTeX macros can selectively render a green verification checkmark beside verified Digital Object Identifiers (DOIs) or trigger build-time compiler warnings for unverified or dead links.

2. Open Access & Licensing Terms (`url+an`, `file+an`)

Context Theses submitted to DiVA often must comply with funder mandates (*e.g.,* Plan S) requiring explicit distinction between closed publisher links, author-accepted manuscripts (AAM), and open-access mirrors.

Listing 25: Syntax for Open Access & Licensing Terms

Syntax

```
url      = {https://doi.org/10.1016/j.comnet
   ↪ .2025.109876},
url+an   = {access="closed"; version="vor"},
url:mirror = {https://urn.kb.se/resolve?urn=
   ↪ urn:nbn:se:kth:diva-345678},
url:mirror+an = {access="gold"; license="cc-
   ↪ by-4.0"; version="aam"}
```

Application The bibliography driver can conditionally typeset open-access badges, Creative Commons micro-icons, or repository links.

3. Publication Venue Integrity & Indexing (`journal+an`, `booktitle+an`)

Context Academic reporting increasingly requires tracking journal quartiles (Q1/Q2), Scopus/Web of Science indexing, or conference CORE rankings (A*, A, B).

Listing 26: Syntax for Publication Venue Integrity & Indexing

Syntax

```
journal     = {IEEE Transactions on
   ↪ Networking},
journal+an = {issn="1063-6692"; ranking="
   ↪ core-a"; jcr="q1"; sjr="1.82"},
booktitle  = {Proceedings of ACM SIGCOMM
   ↪ '25},
booktitle+an = {ranking="core-a-star";
   ↪ acceptance_rate="15.2%"}
```

Application A student can generate extended bibliographic summaries (or committee review appendices) that automatically display venue selectivity and impact metrics alongside included thesis publications.

4. Funding Acknowledgment & Grant Tracking (`note+an`, `addendum+an`)

Context Doctoral projects frequently span multiple funding bodies (*e.g.,* VR, Vinnova, EU Horizon Europe) where specific publications must trace back to concrete grant IDs.

Listing 27: Syntax for Funding Acknowledgment & Grant Tracking

Syntax

```
addendum    = {Supported by the Swedish
   ↪ Research Council},
addendum+an = {funder="VR"; grant="
   ↪ 2023-04567"; project="6G-SEC"}
```

Application Citation drivers can automatically inject formal grant disclosures into the printed thesis bibliography or export structured funding metadata into DiVA/Metadata Object Description Schema (MODS) exports.

5. Editorial Curation & Retraction Watches (`entry+an`)

Context Long-lived literature repositories occasionally contain retracted papers, expressions of concern, or errata.

Listing 28: Syntax for Editorial Curation & Retraction Watches

Syntax

```
entry+an = {
  retraction_status="clean";
  verified_retractionwatch="2026-08-10";
  curation_notes="Benchmark baseline
   ↪ verified against source code"
}
```

Application Pre-flight CI actions can check Crossref's "is-retracted" API (or other source of retractions) and inject warnings into the PDF during compilation if an author inadvertently cites retracted work.

6. Typographic & Pagination Overrides (`pages+an`, `edition+an`)

Context Older conference proceedings, electronic article numbering (e-locators), or bilingual editions often require localized presentation strings without modifying the raw page numbering data.

Listing 29: Syntax for Typographic & Pagination Overrides

Syntax

```
pages    = {45--60},
pages+an = {type="article-number"; prefix="
   ↪ art: "},
edition  = {2},
edition+an = {swedish="2:a uppl."; english="
   ↪ 2nd ed."}
```

Application Custom formatting drivers can switch between language-sensitive labels without maintaining multiple `.bib` variants.

Because reference managers routinely discard or corrupt non-standard fields, isolating these annotations inside `references-annotations.bib` and merging them via Python guarantees that extended provenance, venue, and licensing data remain permanently intact.

2.9 Contrasting annotations with current workflows

The current design approaches publication management and thesis inclusion from an architectural standpoint that is fundamentally different from a purely in-TEX data-annotation model.

Rather than overloading `.bib` entries with document-assembly directives, the current workflows decouple bibliographic citation data from administrative pipeline state:

External Source of Truth (`publications_map.json`)
: In the current workflow, inclusion (`status: "included"`), label assignments (`paper:A`, `techreport:A`), tab order (`tab_index`), physical reprint paths (`file_path`), page slices (`pdf_pages`), scaling factors, and reproduction copyright notices (`permission_text`) live entirely inside `publications_map.json`. This separates immutable bibliographic facts (which belong in `references.bib`) from the mutable, student-curated decisions about how physical PDF reprints are laid out in a compilation thesis.

Two-Way Upstream Sync (DiVA Discovery)
: The automated workflow pulls publication metadata directly from DiVA via the student's KTHID through GitHub Actions. In a pure in-TEX annotation workflow, the candidate must hand-annotate raw `.bib` files. In the current pipeline, the GitHub Action generates `lib/publications_generated.tex` and `lib/publications_dividers_generated.tex` dynamically from DiVA and `publications_map.json`, removing the manual editing step entirely.

CI/CD Build-Time Validation
: Because paper inclusion is tracked in JSON, the GitHub Actions can validate physical PDF assets **before** typesetting:

`"status": "included", "pdf_downloaded": false` ⟶ Emits visual missing-file warning page
`"status": "included", "pdf_downloaded": true` ⟶ Injects divider tab and binds reprint PDF

: Testing file existence, checking scaling, and enforcing page intervals are trivial in Python, whereas executing robust filesystem checks and multi-page PDF inclusions dynamically inside macro-level `biblatex` hooks is notoriously fragile.

Granular Multi-Author & Collaboration CReDiT
: Handling of massive collaborations (such as, the BNL Multi-Particle Spectrometer example) via `specific_contributors` demonstrates why the wizard approach (`CReDiT_Matrix_Wizard.py`) is cleaner

than annotating `.bib` files. In large multi-author experiments, attempting to attach 14 CReDiT part annotations to individual authors inside a standard BibTeX string quickly becomes unwieldy. Generating a standalone `lib/thesis_contributions_generated.tex` file keeps the bibliography clean while producing an explicit, readable contribution matrix for the defense committee.

The existing pipeline positions the `.bib` file as a traditional citation store, delegating document assembly, inclusion filtering, DiVA harvesting, and PDF insertion to external Python and CI automation where it is most maintainable.

2.10 Support for Mega-Collaborations

To prevent `biber` and LaTeX from crashing on mega-collaborations while still providing granular CReDiT attribution for a student, you can use **sparse target indexing**, (*i.e.,* you only need to add the relevant additional author details):

Listing 30: Example of a specific contributor to a large collaboration

```
@techreport{bnlMPS1975,
  author     = {Lindenbaum, S. J. and {MPS Collaboration}},
  author+an  = {
    3:specific-contributor="Maguire Jr., Gerald Q.";
    3:software="lead";
    3:visualization="lead";
  },
  title      = {The {Brookhaven} {Multi-Particle} {
   ↪ Spectrometer} ({MPS}) Facility},
  year       = {1975},
  number     = {BNL-20202},
  url        = {https://www.osti.gov/servlets/purl
   ↪ /4162299},
  note       = {An invited paper presented at the IV
   ↪ International Symposium on NN Interaction, Syracuse
   ↪ University, May 2-4, 1975.},
  entry+an   = {
    diva_id="bnlMPS1975";
    contribution_note="Summer student working on PDP-10
   ↪ software and graphical display of data.";
    has_specific_contributors="true";
  },
  institution = {Brookhaven National Laboratory},
  type       = {Report},
}
```

2.11 Integrating annotations with current workflows

The challenge with handling entities like the **MPS Collaboration** in standard `.bib` files is that the individual student or researcher (*e.g.,* Gerald Q. Maguire Jr.) is not listed as an indexed name item in the `author` field (which contains only "Lindenbaum, S. J." and "{MPS Collaboration}"). Consequently, biber's item-indexed annotations (`author+an = { 3:software="supporting"}`) cannot bind to an author index (in this example "3") that does not exist in the primary name list (or might only appear far down the list, *i.e.,* beyond the range of the authors processed).

Under biber, this can be solved cleanly using one of two annotation architectures:

1. Standalone Contributor Data Annotation (`contributors+an`)

   Rather than attempting to attach metadata to the primary `author` field, declare an annotated dummy or user field (such as `contributors+an`). Biber allows data annotations on arbitrary fields, even when the underlying base field is sparse or virtual.

   In your `references-annotations.bib` overlay:

   Listing 31: Example of non-named author collaborator inclusion

```
@techreport{bnlMPS1975,
  % Entry annotation to flag a collaborator inclusion
  entry+an = {
    has_specific_contributors = "true";
    collaboration_name        = "MPS Collaboration";
    contribution_note         = "Summer student
   ↪ working on PDP-10 software and graphical display
   ↪ of data."
  },
  % Granular CReDiT annotations for contributors not
   ↪ named in the author field:
  contributors+an = {
    1:name="Maguire Jr., Gerald Q.";
    1:role="software";
    1:role="visualization";
    1:affiliation="Brookhaven National Laboratory"
  }
}
```

Inside LaTeX / BibLaTeX:

You can query this with:

Listing 32: Standalone Contributor Data Annotation code

```
\hasentryannotation{has_specific_contributors}{%
  \par\smallskip\noindent\textbf{Collaboration Contributor
   ↪ Disclosure:}%
  \par\hspace*{1.5em}\getentryannotation{contribution_
   ↪ note}%
  \par\hspace*{1.5em}\getpartannotation[contributors][
   ↪ default][1]{name}: %
    \textit{\getpartannotation[contributors][default][1]{
   ↪ role}}%
}{}
```

2. Part Annotation on Collaboration Author Index (Sub-part Mapping)

   If the collaboration is listed in the `author` field as a collective corporate entity (for instance, index 2 in `Lindenbaum, S. J. and {MPS Collaboration}`), you can attach the individual contributor directly to that author's index using structured part keys:

Listing 33: Syntax for Part Annotation on Collaboration Author Index (Sub-part Mapping)

```
@techreport{bnlMPS1975,
  author    = {Lindenbaum, S. J. and {MPS
   ↪ Collaboration}},
  author+an = {
    % Roles for Professor Lindenbaum
    1:conceptualization="lead";
    1:supervision="lead";
    % Specific contributor anchored to Author 2 (the
   ↪ Collaboration)
    2:specific-contributor="Maguire Jr., Gerald Q.";
    2:software="supporting";
    2:visualization="supporting";
    2:note="Summer student: PDP-10 assembly-to-FORTRAN
   ↪  conversion & track display"
  }
}
```

How the In-TeX Driver Handles It:

In your existing `creditnames` name formatting driver, when the driver processes Author 2 (`MPS Collaboration`), you can add an interceptor:

Listing 34: Code for handling specific-contributor metadata

```
\newcommand*{\checkspecificcontributors}[1]{%
  % #1 = current author index
  \haspartannotation[author][default][#1]{specific-
  ↪ contributor}{%
    \par\hspace*{1.5em}\textbf{Specific Contributor:} %
      \getpartannotation[author][default][#1]{specific
  ↪ -contributor}%
    \haspartannotation[author][default][#1]{note}{%
      \par\hspace*{3em}\textit{\getpartannotation[author][
  ↪ default][#1]{note}}%
    }{}%
  }{}%
}
```

Python Pipeline Synthesis (Connecting `CReDiT_Matrix_Wizard.py`)
In the Streamlit wizard and GitHub Actions workflow:

1. `publications_map.json` remains the interactive store holding `specific_contributors` and `contribution_note`.

2. When the wizard generates or updates `references-annotations.bib`, the Python script inspects `entry.get("specific_contributors")`.

3. If `specific_contributors` contains entries, Python matches the named collaboration in the author list and writes the corresponding `2:specific-contributor="…"` and role slugs directly into the generated overlay entry.

This preserves standard bibliography listings for external readers while enabling the defense committee to inspect candidate contributions to large institutional consortia.

It should be possible to mine the `publications_map.json` file generated by `CReDiT_Matrix_Wizard.py` to produce an initial `references-annotations.bib` file with the CReDiT annotations for authors.

2.12 Scaling with the number of publications a student has in DiVA

The `publications_map.json` file has the same number of entries as the number of publications (other than the current thesis) that an author already has in DiVA. How large is this likely to be? Table 2 shows how many students (from those who defended in 2025) have a given number of other publications in DiVA (*i.e.,* not counting any third-cycle theses).

From this data, we see that the distribution is heavily right-skewed by outliers with 46 and 405 publications in DiVA. Looking at these, we see that the student with 405 publications in DiVA worked in the ATLAS consortium (thesis title: *Divide and Conquer: A Fine-Grained Look at the Higgs Boson Cross Section in WW Decays Using the ATLAS Experiment*). This means that if they were to identify their contributions to specific papers, the student would have to use the method described in Section 2.10. However, in this case, the student wrote a monograph and not a compilation-style thesis. In the case of the student with 46 publications in Electromagnetic Engineering (thesis title: *Modeling and design of lenses combined with array antennas in the millimetre-wave regime*), the student chose to prune, and they included seven publications in their compilation thesis and cited five others that were related.

Of the $N = 310$ theses (comprising both doctoral and licentiate degrees), the median is 7 prior publications, with an interquartile range (IQR) spanning 4 to 10. Less than 2 % of students have 36 or more publications in DiVA, and only 4.2 % have 25 or more. Therefore, the manual effort required to designate which publications are to be included in a compilation thesis* and specify their tab order and logical labels remains feasible. Furthermore, because authors include only a curated subset of their total output, subsequent role curation in `CReDiT_Matrix_Wizard.py` remains a modest-sized task.

*Unfortunately, DiVA does not provide an automated mechanism to query constituent paper linkages for third-cycle theses in aggregate. Consequently, determining the exact number of publications formally incorporated into each compilation thesis requires inspecting each individual record or PDF. Section 2.12.1 examines manually collected data for the year 2025.

Table 2: Histogram of the number of authors with a given number of publications in DiVA for third-cycle students who presented in 2025.

Number of other publications in DiVA	Number of authors
1	6
2	15
3	24
4	31
5	28
6	22
7	31
8	27
9	14
10	17
11	15
12	14
13	12
14	11
15	9
16	5
17	5
18	2
19	3
21	1
22	1
24	4
25	5
26	1
27	1
28	1
34	1
36	2
46	1
405	1

2.12.1 Summary of Included Publications in Compilation theses (2025 Cohort)

Out of the 310 total third-cycle theses presented in 2025, exactly 298 (96.1 %) were compilation-style theses with constituent publications, leaving 12 monographs. The number of included publications in these compilation-style theses from 2025 was manually collected from DiVA ($N = 298$, comprising 271 doctoral and 27 licentiate theses). Table 3 gives the distribution of the number of included publications, while the summary below outlines key statistical measures:

Combined Cohort ($N = 298$) Mean: 4.89; Standard Deviation (s, sample/pop): 1.66 / 1.65; Median: 5.0; Mode: 4 (110 theses); Range: 2 to 12; $Q_1 = 4.0$, $Q_3 = 6.0$, IQR = 2.0.

Doctoral Theses ($N = 271$) Mean: 5.13; Standard Deviation (s, sample/pop): 1.53 / 1.53; Median: 5.0; Mode: 4 (109 theses); Range: 2 to 12; $Q_1 = 4.0$, $Q_3 = 6.0$, IQR = 2.0.

Licentiate Theses ($N = 27$) Mean: 2.48; Standard Deviation (s, sample/pop): 0.58 / 0.57; Median: 2.0; Mode: 2 (15 theses); Range: 2 to 4; $Q_1 = 2.0$, $Q_3 = 3.0$, IQR = 1.0.

Table 3: Distribution of included publications in third-cycle compilation theses presented in 2025.

	Doctoral ($N = 271$)		**Licentiate** ($N = 27$)		**Combined** ($N = 298$)	
Publications	**Freq.**	**%**	**Freq.**	**%**	**Freq.**	**%**
2	4	1.48	15	55.56	19	6.38
3	6	2.21	11	40.74	17	5.70
4	109	40.22	1	3.70	110	36.91
5	66	24.35	–	–	66	22.15
6	43	15.87	–	–	43	14.43
7	24	8.86	–	–	24	8.05
8	9	3.32	–	–	9	3.02
9	5	1.85	–	–	5	1.68
10	2	0.74	–	–	2	0.67
11	2	0.74	–	–	2	0.67
12	1	0.37	–	–	1	0.34

2.12.2 Key Analytical Takeaways With Respect to Included Publications

Compact Scaling Envelope Over 80.4 % of doctoral compilation theses include between 4 and 6 papers, with the modal value being exactly 4.

Licentiate Norm Over 96.3 % of licentiates incorporate either 2 or 3 papers.

CReDiT Tooling Workload Even for the single extreme doctoral outlier with 12 included works, populating `publications_map.json` and managing contributor attributions in `CReDiT_Matrix_Wizard.py` involves an input matrix of modest dimensions. For most doctoral candidates ($Q_1 = 4$, $Q_3 = 6$, IQR $= 2$), the manual CReDiT assignment task requires processing only 4 to 6 rows.

3 Recommendations for generating fake BibLaTeX entries

ORCID iDs are a designated subset of International Standard Name Identifiers (ISNIs)*, specifically within the range `0000-0001-5000-0007` to `0000-0003-5000-0001`. Values entirely outside this assignment block — such as those starting with all zeroes (`0000-0000-XXXX-XXXX`) — do not represent valid ORCID registry allocations. Additionally, ORCID iDs use a MOD 11-2 checksum as the final character; thus, any 16-digit string with a failed checksum is invalid and will be rejected by any schema or API validator. Because valid ISNIs exist outside the designated ORCID block, arbitrary non-ORCID ISNIs should still be avoided to prevent collisions with real registered entities in the global ISNI database; test entries should strictly employ mathematically invalid checksums or zero-filled blocks (*e.g.,* `0000-0000-0000-000X`).

The International DOI Foundation (IDF) has reserved a designated DOI prefix specifically for testing, documentation, and development, and a number of other DOI prefixes are used/reserved for similar purposes. These are:

- `10.1000` (The Demonstration & Documentation Prefix): The *DOI Handbook* explicitly assigns the prefix `10.1000` for examples, demonstration systems, and documentation (similar to example domain names like "`example.com`"). Identifiers using this prefix — such as `10.1000/182` — are reserved for instructional and testing purposes and will not resolve to real production literature.

- DataCite Test Environment (`10.5072`): DataCite reserves prefix `10.5072` strictly for sandbox environments and test accounts. DOIs registered under `10.5072` are test records used during API integration and are never routed to the production registry.

- Crossref Test Prefix (`10.5555`): Crossref provides test accounts with prefixes such as `10.5555` (and similar designated sandbox prefixes) for verifying Extensible Markup Language (XML) deposits and validation pipelines without creating permanent scholarly records.

- Reserved Registrant Code `10.0000`: The `10.0000` space is unassigned to

*See `https://isni.org/` and International Standard Organization (ISO) 27729 [4] for more information.

any DOI Registration Agency (RA) and is invalid for production publication metadata.

For mock datasets, documentation examples, or pipeline test suites, `10.1000` is the standard, RFC/handbook-compliant choice. Use this prefix for examples.

3.1 Authors, Titles, and other Metadata Fields

When constructing synthetic entries for documentation, testing, or pipeline verification, all accompanying fields must be constructed so that they cannot be conflated with genuine scholarly assets:

Explicit Metadata Markers (`placeholder`)

When real names, institutional affiliations, or historical works are used in illustrative examples, the entry should explicitly declare:

```
placeholder = {Placeholder fake reference for
   ↪ use in examples},
```

This provides a machine-readable flag that downstream CI verification scripts (*e.g.,* `bib_cleanup.py`) and pre-flight linters can inspect to ensure synthetic records are not inadvertently retained in final thesis submissions or uploaded to production repositories like DiVA.

Authors & Consortia

Avoid realistic common names that could collide with actual researchers in databases such as Scopus, Web of Science, or DiVA. Prefer conventional metasyntactic names (*e.g.,* "Alice Example" and "Bob Sample") or clearly designated institutional placeholders (*e.g.,* "Researcher, Mock A." or `{{The Example Collaboration}}`).

Titles & Publication Venues

Titles should explicitly include the terms "Example", "Mock", or "Demonstration" (*e.g., A Synthetic Benchmark for Example Systems*). Journals and book titles should use explicitly fictional titles (*e.g., Journal of Illustrative Studies*, *Proceedings of the Demonstration Conference*) to avoid collisions with predatory or newly minted real journals.

ISBNs & ISSNs Follow ISO 2108 recommendations for example International Standard Book Numbers (ISBNs) [5]. Use the standard demonstration ISBN-13 (`978-3-16-148410-0`) or an unassigned/failed-checksum sequence (*e.g.,* `978-0-00-000000-0`). For serial publications, the dummy International Standard Serial Number (ISSN) `0000-0000` is unallocated by the ISSN International Centre.

Domains & URIs Under Request for Comments (RFC) 2606 [6] and its update RFC 6761 [7], only the reserved second-level domains (`example.com`, `example.org`, `example.edu`) or the reserved top-level domains (`.test`, `.example`, `.invalid`) should be used for Uniform Resource Locators (URLs) and repository mirrors. For national repository handles, use zero-padded identifiers (*e.g.,* `urn:nbn:se:kth:diva-000000`).

4 Overleaf Automation

Section 4.1 examines how a script could be used to check for placeholders. Section 4.2 generalizes this approach to copy files from the compilation container to the Overleaf project, so that artifacts can become persistent and potentially be stored in a GitHub repository.

4.1 Automating the Placeholder check

Overleaf's internal build pipeline is fundamentally stateless with respect to the git tree, as the compiler container writes auxiliary files, ".log", and ".pdf" to an ephemeral build cache outside the git repository directory. The web User Interface (UI) exposes these via internal project endpoints that are not exposed in standard git syncs.

Breaking the Software as a Service (SaaS) walled gardens via the UNIX philosophy: **If a browser button triggers an action, there is an underlying HTTP endpoint or a DOM event that can be automated.**

There are two primary approaches to automating this retrieval without manually clicking through the GUI:

Method 1 Use a script to compile the file via Overleaf, then fetch the output.log file and filter it

If we assume that you are logged in and accessing your Overleaf project using Chrome as your browser. You should set the environment variable `OVERLEAF_PROJECT_ID` to be the identifier of your Overleaf project, *i.e.,* the string that follows "project/". Listing 35 shows a Python program that accesses Overleaf (via its Common LaTeX Service Infrastructure (CLSI)), authenticating with your standard session cookie (`overleaf_session2`). Listing 36 shows the output of the program when accessing this LaTeX file. The hardest part of developing this program was programmatically discovering all MongoDB 24-hex ObjectIds. The program can take one of three arguments (see `resolve_file_target`): (*i*) the 24-hexadecimal ID, (*ii*) the exact path name, or (*iii*) part of a path name. If the program finds unresolved placeholders, it returns 2 so it can be used in a CI process.

Listing 35: Use Python to access the build log

```
#!/usr/bin/env python3
"""
fetch_overleaf_check_for_unresolved_placeholders4.
   ↪ py

Automates Overleaf log retrieval and audits for
   ↪ unresolved BibLaTeX placeholder
references. Programmatically discovers all MongoDB
   ↪ 24-hex ObjectIds directly
from Overleaf's /updates and /entities endpoints
   ↪ without requiring manual
JSON mapping files or browser developer window
   ↪ scripts.
"""

import os
import re
import sys
import uuid
import requests
```

```
try:
    import browser_cookie3
except ImportError:
    browser_cookie3 = None

TARGET_BANNER = "UNRESOLVED PLACEHOLDER REFERENCES
    ↪ DETECTED IN THIS BUILD:"
PLACEHOLDER_MARKER = "Unresolved placeholder
    ↪ reference"

FallBack_Project_ID="000000000000000000000000" #
    ↪ replace with your project ID
FallBack_CLSIserver="clsi-cache-zone-b-prod-3" #
    ↪ replace with your CLSI server


def get_authenticated_session(project_id: str) ->
    ↪ tuple[requests.Session, str]:
    session = requests.Session()
    session.headers.update({
        "User-Agent": (
            "Mozilla/5.0 (X11; Linux x86_64)
    ↪ AppleWebKit/537.36 "
            "(KHTML, like Gecko) Chrome/154.0.0.0
    ↪ Safari/537.36"
        ),
        "Referer": f"https://www.overleaf.com/
    ↪ project/{project_id}",
        "Origin": "https://www.overleaf.com",
        "Accept": "application/json, text/plain,
    ↪ */*",
    })

    cookie_val = os.environ.get("
    ↪ OVERLEAF_SESSION_COOKIE")

    if not cookie_val and browser_cookie3:
        try:
            cj = browser_cookie3.chrome(domain_name
    ↪ =".overleaf.com")
```

```
            session.cookies.update(cj)
            for c in cj:
                if c.name == "overleaf_session2":
                    cookie_val = c.value
                    break
        except Exception as e:
            sys.stderr.write(f"Note:
↪ browser_cookie3: {e}\n")

    if cookie_val:
        if "overleaf_session2=" in cookie_val:
            cookie_val = cookie_val.split("
↪ overleaf_session2=")[1].split(";")[0].strip()
        session.cookies.set("overleaf_session2",
↪ cookie_val, domain=".overleaf.com", path="/")

    res = session.get(f"https://www.overleaf.com/
↪ project/{project_id}")
    if res.status_code == 403 or "login" in res.url:
        sys.stderr.write("Authentication failed.
↪ Ensure you are logged into Overleaf in Chrome
↪ .\n")
        sys.exit(1)

    csrf_token = None
    for pat in [
        r'window\.csrfToken\s*=\s*["\']([^"\']+)
↪ ["\']',
        r'<meta\s+name=["\']ol-csrfToken["\']\s+
↪ content=["\']([^"\']+)["\']',
        r'"_csrf"\s*:\s*["\']([^"\']+)["\']',
    ]:
        m = re.search(pat, res.text)
        if m:
            csrf_token = m.group(1)
            break

    if not csrf_token:
        sys.stderr.write("Failed to retrieve CSRF
↪ token from project page.\n")
```

```
        sys.exit(1)

    return session, csrf_token


def fetch_project_doc_mapping(session: requests.
    ↪ Session, project_id: str, csrf_token: str) ->
    ↪ dict[str, str]:
    """
    Queries Overleaf's /updates endpoint to
    ↪ dynamically extract the mapping between
    file relative paths and their 24-hex MongoDB
    ↪ ObjectIds.
    """
    mapping = {}
    headers = {
        "X-Csrf-Token": csrf_token,
        "X-Requested-With": "XMLHttpRequest",
        "Accept": "application/json",
    }

    try:
        r = session.get(f"https://www.overleaf.com/
    ↪ project/{project_id}/updates", headers=
    ↪ headers, timeout=15)
        if r.status_code == 200:
            data = r.json()
            updates = data.get("updates", [])
            for u in updates:
                # Overleaf update records store
    ↪ pathnames and document/blob metadata
                pathnames = u.get("pathnames", [])

                # Check for explicit docs/files
    ↪ lists inside updates
                docs = u.get("docs", []) or u.get("
    ↪ meta", {}).get("docs", [])
                for d in docs:
                    d_id = d.get("id") or d.get("
    ↪ _id")
```

```
                    d_path = d.get("pathname") or d
↪ .get("path")
                    if d_id and d_path:
                        clean_p = d_path.lstrip("/"
↪ )
                        mapping[clean_p] = d_id
                        mapping[os.path.basename(
↪ clean_p)] = d_id

                # Inspect patches/ops where doc IDs
↪  are paired with paths
                patches = u.get("patches", [])
                for p in patches:
                    d_id = p.get("doc") or p.get("
↪ doc_id")
                    d_path = p.get("pathname") or p
↪ .get("path")
                    if d_id and d_path:
                        clean_p = d_path.lstrip("/"
↪ )
                        mapping[clean_p] = d_id
                        mapping[os.path.basename(
↪ clean_p)] = d_id

                # If updates associate a single
↪ doc_id with pathnames
                u_doc_id = u.get("doc") or u.get("
↪ doc_id") or u.get("id")
                if u_doc_id and re.fullmatch(r"[a-f0
↪ -9]{24}", str(u_doc_id)):
                    for path in pathnames:
                        clean_p = path.lstrip("/")
                        mapping[clean_p] = u_doc_id
                        mapping[os.path.basename(
↪ clean_p)] = u_doc_id

            # Fallback regex scan over the payload
↪ for any "pathname":"...", "id":"..."
↪ structures
            if not mapping:
```

```
                for match in re.finditer(
                    r'(?:pathname|path)["\']?\s*:\
    ↪ s*["\']([^"\']+)["\'].*?(?:id|_id|doc)["\']?\
    ↪ s*:\s*["\']([a-f0-9]{24})["\']',
                    r.text
                ):
                    p, did = match.group(1).lstrip(
    ↪ "/"), match.group(2)
                    mapping[p] = did
                    mapping[os.path.basename(p)] =
    ↪ did
    except Exception as e:
        sys.stderr.write(f"Warning: Failed to fetch
    ↪  updates mapping: {e}\n")

    return mapping


def fetch_project_paths(session: requests.Session,
    ↪ project_id: str, csrf_token: str) -> list[str]:
    """Retrieves canonical paths from the /entities
    ↪  endpoint."""
    headers = {
        "X-Csrf-Token": csrf_token,
        "X-Requested-With": "XMLHttpRequest",
        "Accept": "application/json",
    }
    try:
        r = session.get(
            f"https://www.overleaf.com/project/{
    ↪ project_id}/entities?projectId={project_id}",
            headers=headers,
            timeout=10
        )
        if r.status_code == 200:
            data = r.json()
            entities = data.get("entities", [])
            return [e.get("path", "").lstrip("/") for
    ↪  e in entities if "path" in e]
    except Exception:
```

```
        pass
    return []


def resolve_file_target(
    arg: str | None,
    mapping: dict[str, str],
    known_paths: list[str],
) -> tuple[str | None, str | None]:
    """Resolves an input argument to (root_doc_id,
    ↪ root_resource_path)."""
    if not arg:
        return None, None

    clean = arg.strip().lstrip("./")

    # Invert mapping to support doc_id -> canonical
    ↪  path
    id_to_path = {}
    for p, did in mapping.items():
        if "/" in p:
            id_to_path[did] = p
    for p, did in mapping.items():
        id_to_path.setdefault(did, p)

    # 1. Direct 24-character hexadecimal MongoDB
    ↪ ObjectId
    if re.fullmatch(r"[a-f0-9]{24}", clean):
        doc_id = clean
        path = id_to_path.get(doc_id)
        if not path:
            path = next((p for p in known_paths if os.
    ↪ path.basename(p) == clean), None)
        return doc_id, path

    # 2. Exact match in mapping
    base = os.path.basename(clean)
    doc_id = mapping.get(clean) or mapping.get(base
    ↪ )
```

```
    # 3. Substring / fuzzy match across mapped
    ↪ paths and known entities
    if not doc_id:
        candidates = {k: v for k, v in mapping.items
    ↪ () if base.lower() in k.lower()}
        unique_ids = set(candidates.values())
        if len(unique_ids) == 1:
            doc_id = next(iter(unique_ids))
            clean = max([k for k in candidates if
    ↪ candidates[k] == doc_id], key=len)
        elif len(unique_ids) > 1:
            sys.stderr.write(f"Ambiguous match for
    ↪ '{arg}':\n")
            for k in sorted(candidates.keys()):
                sys.stderr.write(f"  * {k}\n")
            sys.exit(1)

    # 4. Resolve relative canonical directory path
    full_path = id_to_path.get(doc_id)
    if not full_path:
        for p in known_paths:
            if os.path.basename(p) == base or p ==
    ↪ clean:
                full_path = p
                break

    if not full_path:
        full_path = clean if "/" in clean else base

    return doc_id, full_path


def fetch_latest_log(
    session: requests.Session,
    project_id: str,
    csrf_token: str,
    root_doc_id: str | None = None,
    root_resource_path: str | None = None,
) -> str:
    compile_url = f"https://www.overleaf.com/
```

```
↪ project/{project_id}/compile?
↪ enable_pdf_caching=true"
 headers = {
     "Content-Type": "application/json",
     "X-Csrf-Token": csrf_token,
 }

 payload = {
     "draft": False,
     "png2pdf": True,
     "incrementalCompilesEnabled": True,
     "stopOnFirstError": False,
     "editorId": str(uuid.uuid4()),
 }

 # Pass the resolved MongoDB ObjectId and
↪ canonical resource path
 if root_doc_id:
     payload["rootDoc_id"] = root_doc_id
 if root_resource_path:
     payload["rootResourcePath"] =
↪ root_resource_path

 # Only request a silent check when building the
↪  default root document
 if not root_doc_id and not root_resource_path:
     payload["check"] = "silent"

 compile_res = session.post(compile_url, headers
↪ =headers, json=payload)
 if compile_res.status_code != 200:
     sys.stderr.write(f"POST compile failed:
↪ HTTP {compile_res.status_code}\n")
     sys.stderr.write(compile_res.text[:300] + "
↪ \n")
     sys.exit(1)

 data = compile_res.json()
 output_files = data.get("outputFiles", [])
```

```
    log_entry = next((f for f in output_files if f.get
↪ ("path") == "output.log"), None)
    if not log_entry:
        sys.stderr.write("No output.log found in
↪ compile artifacts.\n")
        sys.stderr.write(f"Compile status: {data.
↪ get('status')}\n")
        sys.exit(1)

    params = {
        "clsiserverid": data.get("clsiServerId",
↪ FallBack_CLSIserver),
        "compileGroup": data.get("compileGroup", "
↪ priority"),
        "editorId": payload["editorId"],
    }

    cdn_domain = data.get(
        "pdfDownloadDomain", "https://compiles.
↪ overleafusercontent.com"
    ).rstrip("/")

    # Primary URL directly to Overleaf usercontent
↪ CDN
    cdn_url = f"{cdn_domain}{log_entry['url']}"
    res = session.get(cdn_url, params=params)
    if res.status_code == 200:
        return res.text

    # Fallback to main host
    main_url = f"https://www.overleaf.com{log_entry
↪ ['url']}"
    res = session.get(main_url, params=params)
    if res.status_code == 200:
        return res.text

    sys.stderr.write(f"Failed to fetch output.log (
↪ HTTP {res.status_code}).\n")
    sys.exit(1)
```

```
def audit_placeholders(log_content: str):
    has_marker = PLACEHOLDER_MARKER in log_content
    has_banner = TARGET_BANNER in log_content

    if not (has_marker or has_banner):
        sys.stdout.write("✅ Clean build: No
↪ unresolved placeholder references.\n")
        sys.exit(0)

    wrapped_re = re.compile(
        r"Package biblatex Warning:\s*Unresolved
↪ placeholder reference\s*'(?P<key>[^']+)':\s
↪ *(?P<note>[^\n]+(?:\n[^\n]+)*?\.)(?:\s+(?:
↪ Underfull|Overfull|Package|\n|\[|$))"
    )
    matches = list(wrapped_re.finditer(log_content))

    detected = {}
    if matches:
        for w in matches:
            key = w.group("key")
            note = " ".join(w.group("note").split()
↪ ).rstrip(".")
            detected.setdefault(key, note)
    else:
        for line in log_content.splitlines():
            if PLACEHOLDER_MARKER in line:
                m = re.search(r"reference\s
↪ *'([^']+)':\s*(.*?)(?:\.|$)", line)
                if m:
                    detected.setdefault(m.group(1),
↪  m.group(2).strip().rstrip("."))

    total_occurrences = len(matches) if matches else
↪ len(detected)
    sys.stdout.write(
        f"\❌n UNRESOLVED PLACEHOLDERS DETECTED ({
↪ len(detected)} unique keys, {
↪ total_occurrences} total citations):\n"
```

```
    )
    for key, note in sorted(detected.items()):
        sys.stdout.write(f"  * [{key}] {note}\n")

    sys.stdout.write("\n")
    sys.exit(2)


def main():
    project_id = os.environ.get("
    ↪ OVERLEAF_PROJECT_ID") or FallBack_Project_ID
    target_arg = sys.argv[1].strip() if len(sys.argv
    ↪ ) > 1 else None

    session, csrf_token = get_authenticated_session
    ↪ (project_id)
    known_paths = fetch_project_paths(session,
    ↪ project_id, csrf_token)
    doc_mapping = fetch_project_doc_mapping(session
    ↪ , project_id, csrf_token)

    root_id, resource_path = resolve_file_target(
    ↪ target_arg, doc_mapping, known_paths)

    if target_arg:
        sys.stdout.write(f"Target Resolved:\n  *
    ↪ rootDoc_id: {root_id}\n  * rootResourcePath:
    ↪ {resource_path}\n\n")

    log_text = fetch_latest_log(
        session=session,
        project_id=project_id,
        csrf_token=csrf_token,
        root_doc_id=root_id,
        root_resource_path=resource_path,
    )
    audit_placeholders(log_text)


if __name__ == "__main__":
```

```
    main()
```

Listing 36: Check for remaining placholders

```
./fetch_overleaf_check_for_unresolved_placeholders4
   ↪ .py
   ↪ README_programmer_notes_Biber_and_BibLaTeX.
   ↪ tex
Target Resolved:
  * rootDoc_id: None
  * rootResourcePath: README_notes/
   ↪ README_programmer_notes_Biber_and_BibLaTeX.
   ↪ tex❌


 UNRESOLVED PLACEHOLDERS DETECTED (2 unique keys, 5
   ↪  total citations):
  * [doe2025collaborative] Placeholder fake
   ↪ reference for use in examples
  * [maguire2026pipeline] Placeholder fake
   ↪ reference for use in examples
```

Method 2 Inverting the Flow (Build on GitHub Actions via Git Bridge)

If you already maintain the two-way **Overleaf ⟷ GitHub** sync, there is an alternative that avoids browser reverse-engineering altogether:

(a) When the student clicks “Sync to GitHub” or pushes to GitHub from Overleaf, GitHub receives the raw source tree (“.tex”, “.bib”, graphics, *etc.*).

(b) Rather than extract the log from Overleaf’s proprietary runner, let a **GitHub Actions workflow compile the document natively** using the official TeX Live / LuaLaTeX container (“texlive/texlive:latest”). Listing 37 shows such a compilation workflow.

(c) The GitHub runner has full, unrestricted access to “stdout” and “output.log”.

(d) It executes the check on the spot. An example of the output is shown in Figure 1. Alternatively, it could be arranged such that

if it fails the Action check (putting a red ❌ right on the commit or pull request), and optionally drops an automated comment on the GitHub commit pointing to the unresolved keys.

(e) The workflow compile-lualatex.yml when used on a single compilation of `README_programmer_notes_Biber_and_BibLaTeX.tex` produces two artifacts (the `.log` and `.pdf` files), as shown in Figure 2. These are placed in the **artifacts** folder, so that the user can easily access them.

This keeps Overleaf purely as an editor, while GitHub Actions serves as the canonical verification gate.

Listing 37: Compiling on Gitub with compile-lualatex.yml workflow. Note that some markdown characters required escaping into LaTeX to print them suitably.

```yaml
name: Compile LuaLaTeX Documents
permissions:
  contents: write

on:
  push:
    branches: [ main ]
    paths:
      - '**.tex'
      - '**.bib'
  workflow_dispatch:
    inputs:
      target_doc:
        description: 'Select document to compile (or choose "
        ↪ all")'
        required: true
        default: 'README_notes/
        ↪ README_programmer_notes_Biber_and_BibLaTeX.tex'
        type: choice
        options:
          - 'examplethesis.tex'
          - 'Quick_Start_Guide.tex'
          - 'README_3rd_cycle_author.tex'
```

```
      - 'Generate_titles_in_plain_text.tex'
      - 'tools/listfonts_and_generate_exclusion_list.tex'
      - 'tools/luaotfload-tool-find-font.tex'
      - 'tools/restore_font_cache.tex'
      - 'tools/Saving_and_restoring_font_cache.tex'
      - 'tools/updatefontcache.tex'
      - 'tools/warmup.tex'
      - 'README_notes/README_for_3rd_cycle_version.tex'
      - 'README_notes/README_for_administrators.tex'
      - 'README_notes/README_notes_reduced.tex'
      - 'README_notes/README_programmer_notes_appendix.
    ↪ tex'
      - 'README_notes/
    ↪ README_programmer_notes_Biber_and_BibLaTeX.tex'
      - 'README_notes/README_programmer_notes-
    ↪ JSON_to_MODS.tex'
      - 'README_notes/README_programmer_notes-
    ↪ PDF_metadata.tex'
      - 'README_notes/README_programmer_notes.tex'
      - 'README_notes/README_supervisor_notes.tex'
      - 'README_notes/README_Tekla_template.tex'
      - 'all'

jobs:
  #--
  # Job 1: Compile a single selected document on demand
  #--
  compile-single:
    if: "${{ github.event_name == 'workflow_dispatch' &&
      ↪ github.event.inputs.target_doc != 'all' }}"
    runs-on: ubuntu-latest
    permissions:
      contents: write
    container:
      image: texlive/texlive:latest

    steps:
      - name: Checkout repository
```

```
    uses: actions/checkout@v4
    with:
      ref: ${{ github.ref_name }}
      # Fetch full history so pull/rebase works smoothly
      fetch-depth: 0
      # Ensure GITHUB_TOKEN credentials persist for git
    ↪ push
      persist-credentials: true


  - name: Install System Fonts
    run: |
      apt-get update && apt-get install -y --no-install-recommends \
        fontconfig \
        fonts-noto-core \
        fonts-noto-extra \
        fonts-noto-cjk \
        fonts-noto-cjk-extra \
        fonts-noto-mono
      fc-cache -f
      luaotfload-tool --update

  - name: Compile with LuaLaTeX
    run: |
      TARGET="${{ github.event.inputs.target_doc }}"
      TARGET_DIR=$(dirname "$TARGET")

      OUT_DIR="$GITHUB_WORKSPACE/artifacts"
      mkdir -p "$OUT_DIR"

      cd "$GITHUB_WORKSPACE"
      export TEXINPUTS=".:${GITHUB_WORKSPACE}/${
    ↪ TARGET_DIR}//:${GITHUB_WORKSPACE}//:"
      export BIBINPUTS=".:${GITHUB_WORKSPACE}/${TARGET_DIR
    ↪ }//:${GITHUB_WORKSPACE}//:"

      echo "==> Compiling $TARGET from workspace root"
      latexmk -lualatex -interaction=nonstopmode -synctex=1 \
```

```
        -outdir="$OUT_DIR" "$TARGET"
- name: Audit BibLaTeX Unresolved Placeholders
  run: |
    python3 - << 'EOF'
    import os
    import re
    import sys
    import glob

    TARGET_BANNER = "UNRESOLVED PLACEHOLDER
↪ REFERENCES DETECTED IN THIS BUILD:"
    PLACEHOLDER_MARKER = "Unresolved placeholder
↪ reference"

    # 1. Locate the generated .log file in artifacts/
    log_files = glob.glob("artifacts/**/*.log", recursive=True)
    if not log_files:
        print("::warning::No .log file found in artifacts/
↪  to audit.")
        sys.exit(0)

    log_file = log_files[0]
    print(f"==> Auditing log file: {log_file}")

    with open(log_file, "r", encoding="utf-8", errors="replace") as
↪ f:
        log_content = f.read()

    has_marker = PLACEHOLDER_MARKER in log_content
    has_banner = TARGET_BANNER in log_content

    summary_path = os.environ.get("GITHUB_STEP_SUMMARY")

    if not (has_marker or has_banner):
        msg = f"✅ Clean build: No unresolved placeholder
↪ references in `{os.path.basename(log_file)}`."
        print(msg)
```

```
        if summary_path:
            with open(summary_path, "a", encoding="utf-8") as sf:
                sf.write(f"### 🔍 BibLaTeX Placeholder Audit\n\n
↪ {msg}\n\n")
        sys.exit(0)

    # Multiline wrapper regex from audit_placeholders
↪ ()
    wrapped_re = re.compile(
        r"Package biblatex Warning:\s*Unresolved
↪ placeholder reference\s*'(?P<key>[^']+)':\s*(?P<
↪ note>[^\n]+(?:\n[^\n]+)*?\.)(?:\s+(?:Underfull|
↪ Overfull|Package|\n|\[|$))"
    )
    matches = list(wrapped_re.finditer(log_content))

    detected = {}
    if matches:
        for w in matches:
            key = w.group("key")
            note = "  ".join(w.group("note").split()).rstrip(" .")
            detected.setdefault(key, note)
    else:
        for line in log_content.splitlines():
            if PLACEHOLDER_MARKER in line:
                m = re.search(r"reference\s*'([^']+)':\s*(.*?)
↪ (?:\.|$)", line)
                if m:
                    detected.setdefault(m.group(1), m.group(2).strip().rstrip(" .
↪ "))

    total_occurrences = len(matches) if matches else len(detected)
    unique_keys_count = len(detected)

    # Output to console
    print(f"\❌n UNRESOLVED PLACEHOLDERS DETECTED ({
↪ unique_keys_count} unique keys, {
```

```
↪ total_occurrences} total citations):")
  for key, note in sorted(detected.items()):
    print(f"  * [{key}] {note}")
    # Emit native GitHub Actions warning annotations
    print(f"::warning file={log_file}::Unresolved
↪ placeholder reference [{key}]: {note}")

  # Write rich Markdown report to the GitHub Actions
↪  Job Summary
  if summary_path:
    with open(summary_path, "a", encoding="utf-8") as sf:
      sf.write(f"### ⚠️ BibLaTeX Unresolved Placeholders
↪  Detected in `{os.path.basename(log_file)}`\n\n")
      sf.write(f"- **Unique Keys:** {unique_keys_count}\
↪ n")
      sf.write(f"- **Total Occurrences:** {
↪ total_occurrences}\n\n")

      # Use HTML table markup for guaranteed column
↪ widths and no-wrap behavior
      sf.write("<table>\n")
      sf.write("  <thead>\n")
      sf.write("    <tr>\n")
      sf.write('      <th align="left" style="width:
↪ 30%; white-space: nowrap;">Citation Key</th>\n')
      sf.write('      <th align="left">Resolution Note
↪ </th>\n')
      sf.write("    </tr>\n")
      sf.write("  </thead>\n")
      sf.write("  <tbody>\n")
      for key, note in sorted(detected.items()):
        clean_key = key.strip()
        clean_note = note.strip()
        sf.write("    <tr>\n")
        sf.write(f'      <td style="white-space: nowrap
↪ ;"><code>{clean_key}</code></td>\n')
        sf.write(f"      <td>{clean_note}</td>\n")
```

```
            sf.write("      </tr>\n")
        sf.write("    </tbody>\n")
        sf.write("</table>\n\n")

    # Exit with code 0 to allow artifact committing/
↪ uploading,
    # or use sys.exit(2) if you want the CI workflow
↪ run to fail when placeholders exist.
    sys.exit(0)
    EOF

- name: Commit artifacts to repository
  run: |
    cd "$GITHUB_WORKSPACE"

    # 1. Trust workspace UID
    git config --global --add safe.directory "$GITHUB_WORKSPACE"
    git config --global user.name "github-actions[bot]"
    git config --global user.email "github-actions[bot]@users.
↪ noreply.github.com"

    # 2. Stage only PDFs, bibs, and json files inside
↪ artifacts/
    find artifacts -type f \( -name "*.pdf" -o -name "*.bib" -o -
↪ name "*.json" -o -name "*.log" \) -print0 | xargs -0 -r git add
↪  -f

    # 3. Commit only if changes were detected
    if ! git diff --cached --quiet; then
      echo "==> Changes detected, committing..."
      git commit -m "docs: automated build artifacts for $
↪ {{ github.event.inputs.target_doc }} [skip ci]"
      git pull --rebase origin "${{ github.ref_name }}"
      git push origin "HEAD:${{ github.ref_name }}"
      echo "Artifacts pushed to branch ${{ github.
↪ ref_name }}."
    else
      echo "No artifact changes to commit."
```

```
      fi

    - name: Upload Build Artifacts
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: artifacts-single
        path: |
          artifacts*/**/*.pdf
          artifacts*/**/*.log
          artifacts*/**/*.bib
          artifacts*/**/*.json
        if-no-files-found: warn

#--
# Job 2: Compile all defined documents in parallel (
  ↪ push or dispatch 'all')
#--
compile-matrix:
  if: "${{ (github.event_name == 'workflow_dispatch' &&
    ↪ github.event.inputs.target_doc == 'all') }}" #
    ↪ github.event_name == 'push' ||
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false
    matrix:
      include:
        - path: 'examplethesis.tex'
          slug: "examplethesis"
        - path: 'Quick_Start_Guide.tex'
          slug: "Quick_Start"
        - path: 'README_3rd_cycle_author.tex'
          slug: "3rd_cycle_author"
        - path: 'README_notes/README_for_3rd_cycle_version.
    ↪ tex'
          slug: "3rd_cycle_version"
        - path: 'README_notes/README_for_administrators.tex'
          slug: "for_administrators"
```

```
    - path: 'README_notes/README_notes_reduced.tex'
      slug: "notes_reduced"
    - path: 'README_notes/
  ↪ README_programmer_notes_appendix.tex'
      slug: "programmer_notes_appendix"
    - path: "README_notes/
  ↪ README_programmer_notes_Biber_and_BibLaTeX.tex"
      slug: "programmer-notes-Biber"
    - path: 'README_notes/README_programmer_notes-
  ↪ JSON_to_MODS.tex'
      slug: "programmer_notes_JSON"
    - path: 'README_notes/README_programmer_notes-
  ↪ PDF_metadata.tex'
      slug: "programmer_notes_PDF"
    - path: 'README_notes/README_programmer_notes.tex'
      slug: "programmer_notes"
    - path: 'README_notes/README_supervisor_notes.tex'
      slug: "supervisor_notes"
    - path: 'README_notes/README_Tekla_template.tex'
      slug: "Tekla"
    # Add any other root or README_notes targets here:
    # - path: "README_notes/another_note.tex"
    #   slug: "another-note"

container:
  image: texlive/texlive:latest

steps:
  - name: Checkout repository
    uses: actions/checkout@v4

  - name: Install System Fonts
    run: |
      apt-get update && apt-get install -y --no-install-recommends \
        fontconfig \
        fonts-noto-core \
        fonts-noto-extra \
        fonts-noto-cjk \
```

```
      fonts-noto-cjk-extra \
      fonts-noto-mono
    fc-cache -f
    luaotfload-tool --update

- name: Compile ${{ matrix.slug }} with LuaLaTeX
  run: |
    TARGET="${{ matrix.path }}"
    TARGET_DIR=$(dirname "$TARGET")

    OUT_DIR="$GITHUB_WORKSPACE/artifacts-${{ matrix.
↪ slug }}"
    mkdir -p "$OUT_DIR"

    cd "$GITHUB_WORKSPACE"
    export TEXINPUTS=".:${GITHUB_WORKSPACE}/${
↪ TARGET_DIR}//:${GITHUB_WORKSPACE}//:"
    export BIBINPUTS=".:${GITHUB_WORKSPACE}/${TARGET_DIR
↪ }//:${GITHUB_WORKSPACE}//:"

    latexmk -lualatex -interaction=nonstopmode -synctex=1 \
        -outdir="$OUT_DIR" "$TARGET"

- name: Upload Artifacts (${{ matrix.slug }})
  uses: actions/upload-artifact@v4
  with:
    name: "artifacts-${{ matrix.slug }}"
    path: |
      artifacts-${{ matrix.slug }}/*.pdf
            artifacts-${{ matrix.slug }}/*.bib
            artifacts-${{ matrix.slug }}/*.json
          if-no-files-found: warn
```

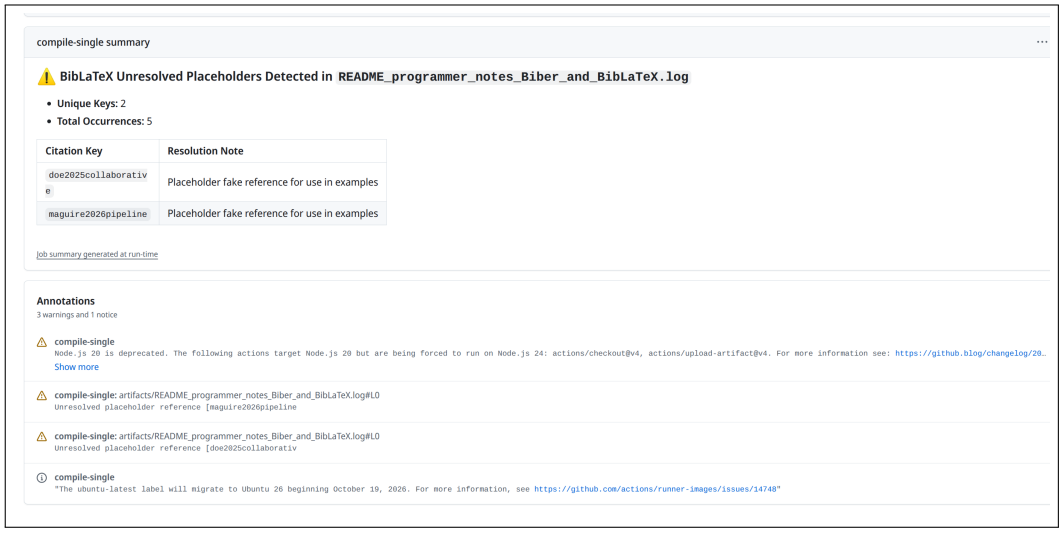


Figure 1: Result of the compilation showing the case of unresolved placeholders

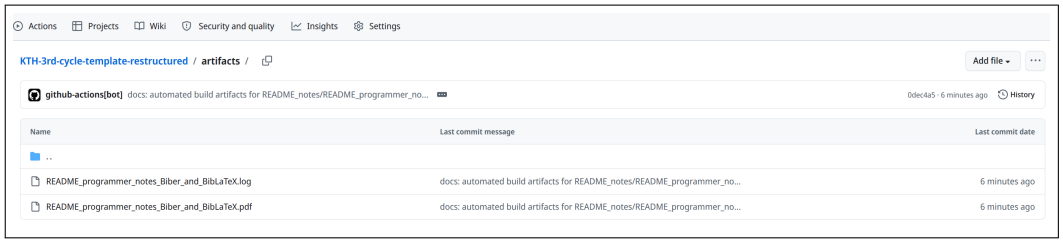


Figure 2: Artifacts created by the workflow compile-lualatex.yml when used on a single compilation of `README_programmer_notes_Biber_and_BibLaTeX.tex`

4.2 Further Automation

It is possible to build other automation on the mechanism of Method 1 shown in Listing 35. For example, Python code shown in Listing 38 can copy files from the build container to the Overleaf project (*i.e.,* making them stable files in the project's directory tree). An example of using this is shown in Listing 39. Further examples include `fordiva.json`, `referencesUsed.bib`, and `citedtags.bib` for a thesis.

Listing 38: Use Python to compile the project and upload a file, such as the PDF. Note that LaTeX commands were added for layout purposes.

```
#!/usr/bin/env python3
"""
copy_files_from_build_to_project.py

Compiles an Overleaf sub-document or project root,
    ↪ downloads generated build
artifacts (e.g. output.pdf, fordiva.json, referencesUsed.
    ↪ bib, citedtags.bib)
from the CLSI build container, and persists them into the
    ↪ project tree.
"""

import io
import os
import re
import sys
import time
import uuid
import requests

try:
    import browser_cookie3
except ImportError:
    browser_cookie3 = None

FallBack_Project_ID="000000000000000000000000" # replace
    ↪ with your project ID
FallBack_CLSIserver="clsi-cache-zone-b-prod-3" # replace
    ↪ with your CLSI server


def get_authenticated_session(project_id: str) -> tuple[
    ↪ requests.Session, str]:
    session = requests.Session()
    session.headers.update({
        "User-Agent": (
            "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit
    ↪ /537.36 "
            "(KHTML, like Gecko) Chrome/154.0.0.0 Safari
    ↪ /537.36"
```

```
        ),
        "Referer": f"https://www.overleaf.com/project/{
↪ project_id}",
        "Origin": "https://www.overleaf.com",
        "Accept": "application/json, text/plain, */*",
    })

    cookie_val = os.environ.get("OVERLEAF_SESSION_COOKIE")
    if not cookie_val and browser_cookie3:
        try:
            cj = browser_cookie3.chrome(domain_name=".
↪ overleaf.com")
            session.cookies.update(cj)
            for c in cj:
                if c.name == "overleaf_session2":
                    cookie_val = c.value
                    break
        except Exception as e:
            sys.stderr.write(f"Note: browser_cookie3: {e}\n
↪ ")

    if cookie_val:
        if "overleaf_session2=" in cookie_val:
            cookie_val = cookie_val.split("
↪ overleaf_session2=")[1].split(";")[0].strip()
        session.cookies.set("overleaf_session2", cookie_val,
↪  domain=".overleaf.com", path="/")

    res = session.get(f"https://www.overleaf.com/project/{
↪ project_id}")
    if res.status_code == 403 or "login" in res.url:
        sys.stderr.write("Authentication failed. Ensure you
↪  are logged into Overleaf in Chrome.\n")
        sys.exit(1)

    csrf_token = None
    for pat in [
        r'<meta\s+name=["\']ol-csrfToken["\']\s+content
↪ =["\']([^"\']+)["\']',
        r'window\.csrfToken\s*=\s*["\']([^"\']+)["\']',
```

```
        r'"_csrf"\s*:\s*["\']([^"\']+)["\']',
    ]:
        m = re.search(pat, res.text)
        if m:
            csrf_token = m.group(1)
            break

    if not csrf_token:
        sys.stderr.write("Failed to retrieve CSRF token
↪ from project page.\n")
        sys.exit(1)

    return session, csrf_token


def fetch_project_paths(session: requests.Session,
↪ project_id: str, csrf_token: str) -> list[str]:
    headers = {
        "X-Csrf-Token": csrf_token,
        "X-Requested-With": "XMLHttpRequest",
        "Accept": "application/json",
    }
    try:
        r = session.get(
            f"https://www.overleaf.com/project/{project_id
↪ }/entities?projectId={project_id}",
            headers=headers,
            timeout=10,
        )
        if r.status_code == 200:
            data = r.json()
            return [e.get("path", "").lstrip("/") for e in
↪ data.get("entities", []) if "path" in e]
    except Exception:
        pass
    return []


def get_or_create_destination_folder(
    session: requests.Session,
```

```
    project_id: str,
    csrf_token: str,
    base_folder_name: str = "artifacts",
) -> tuple[str, str]:
    """
    Creates a destination folder on the project root and
    ↪ returns (folder_name, folder_id).
    If base_folder_name already exists, falls back to a
    ↪ timestamped folder name
    to guarantee a freshly allocated, valid MongoDB
    ↪ ObjectId.
    """
    headers = {
        "X-Csrf-Token": csrf_token,
        "Accept": "application/json",
        "Content-Type": "application/json",
    }

    candidates = [
        base_folder_name,
        f"{base_folder_name}_{int(time.time())}",
    ]

    for fname in candidates:
        post_res = session.post(
            f"https://www.overleaf.com/project/{project_id
    ↪ }/folder",
            headers=headers,
            json={"name": fname},
            timeout=10,
        )
        if post_res.status_code in (200, 201):
            data = post_res.json()
            fid = data.get("_id")
            if fid:
                return fname, fid

    sys.stderr.write(f"Error: Unable to create destination
    ↪ folder in Overleaf.\n")
    sys.exit(1)
```

```
def compile_and_get_artifacts(
    session: requests.Session,
    project_id: str,
    csrf_token: str,
    root_resource_path: str | None = None,
) -> tuple[str, list[dict], dict]:
    compile_url = f"https://www.overleaf.com/project/{
    ↪ project_id}/compile?enable_pdf_caching=true"
    headers = {
        "Content-Type": "application/json",
        "X-Csrf-Token": csrf_token,
    }

    editor_id = str(uuid.uuid4())
    payload = {
        "draft": False,
        "png2pdf": True,
        "incrementalCompilesEnabled": True,
        "stopOnFirstError": False,
        "editorId": editor_id,
    }

    if root_resource_path:
        payload["rootResourcePath"] = root_resource_path
    else:
        payload["check"] = "silent"

    compile_res = session.post(compile_url, headers=headers
    ↪ , json=payload)
    if compile_res.status_code != 200:
        sys.stderr.write(f"POST compile failed: HTTP {
    ↪ compile_res.status_code}\n")
        sys.stderr.write(compile_res.text[:300] + "\n")
        sys.exit(1)

    data = compile_res.json()
    status = data.get("status", "unknown")
    routing = {
```

```
        "clsiServerId": data.get("clsiServerId",
    ↪ FallBack_CLSIserver),
        "compileGroup": data.get("compileGroup", "priority"
    ↪ ),
        "editorId": editor_id,
        "pdfDownloadDomain": data.get("pdfDownloadDomain",
    ↪ "https://compiles.overleafusercontent.com").rstrip("/"
    ↪ ),
    }
    return status, data.get("outputFiles", []), routing


def download_build_artifact(
    session: requests.Session,
    artifact_url: str,
    routing: dict,
) -> bytes | None:
    params = {
        "clsiserverid": routing["clsiServerId"],
        "compileGroup": routing["compileGroup"],
        "editorId": routing["editorId"],
    }

    cdn_url = f"{routing['pdfDownloadDomain']}{artifact_url
    ↪ }"
    res = session.get(cdn_url, params=params)
    if res.status_code == 200:
        return res.content

    main_url = f"https://www.overleaf.com{artifact_url}"
    res = session.get(main_url, params=params)
    if res.status_code == 200:
        return res.content

    return None


def upload_to_folder(
    session: requests.Session,
    project_id: str,
```

```
    csrf_token: str,
    file_bytes: bytes,
    dest_name: str,
    folder_id: str,
) -> bool:
    upload_url = f"https://www.overleaf.com/project/{
    ↪ project_id}/upload"

    # 1. folder_id MUST be in query params, not body
    params = {
        "folder_id": folder_id,
    }

    headers = {
        "X-Csrf-Token": csrf_token,
        "Accept": "application/json",
    }

    # 2. Body must contain ONLY 'name' (FineUploader uses
    ↪ this, validator allows this)
    data = {
        "name": dest_name,
    }

    # 3. File tuple: (filename, stream, mime_type)
    files = {
        "qqfile": (dest_name, io.BytesIO(file_bytes), "
    ↪ application/octet-stream"),
    }

    res = session.post(upload_url, headers=headers, params=
    ↪ params, data=data, files=files)
    if res.status_code in (200, 201):
        try:
            return res.json().get("success", True)
        except Exception:
            return True

    sys.stderr.write(f"Failed to upload '{dest_name}': HTTP
    ↪  {res.status_code} - {res.text[:300]}\n")
```

```
    return False


def main():
    if len(sys.argv) < 2:
        sys.stderr.write(
            "Usage:\n"
            "  ./copy_files_from_build_to_project.py [
↪ subdoc.tex] <artifact1> [artifact2 ...]\n"
            "  ./copy_files_from_build_to_project.py
↪ README_programmer_notes_Biber_and_BibLaTeX.tex output.
↪ pdf fordiva.json referencesUsed.bib citedtags.bib\n\n"
        )
        sys.exit(1)

    project_id = os.environ.get("OVERLEAF_PROJECT_ID") or
↪ FallBack_Project_ID

    args = sys.argv[1:]
    target_subdoc = None

    if args[0].endswith(".tex"):
        target_subdoc = args[0]
        targets = args[1:]
    else:
        targets = args

    if not targets:
        targets = ["output.pdf", "fordiva.json", "
↪ referencesUsed.bib", "citedtags.bib"]

    session, csrf_token = get_authenticated_session(
↪ project_id)
    known_paths = fetch_project_paths(session, project_id,
↪ csrf_token)

    # Dynamically resolve destination folder and ID
    folder_name, folder_id =
↪ get_or_create_destination_folder(session, project_id,
↪ csrf_token, "artifacts")
```

```
 root_resource_path = None
 if target_subdoc:
     clean_target = target_subdoc.strip().lstrip("./")
     root_resource_path = next(
         (p for p in known_paths if p == clean_target or os
↪ .path.basename(p) == os.path.basename(clean_target)),
         clean_target,
     )

 sys.stdout.write(f"Initiating compilation on Overleaf (
↪ target: {root_resource_path or 'default root'})...\n")
 status, artifacts, routing = compile_and_get_artifacts(
     session=session,
     project_id=project_id,
     csrf_token=csrf_token,
     root_resource_path=root_resource_path,
 )

 sys.stdout.write(f"Compilation Status: {status}\n")
 artifact_map = {item.get("path"): item.get("url") for
↪ item in artifacts if "path" in item and "url" in item}
 sys.stdout.write(f"Total build artifacts returned: {len
↪ (artifact_map)}\n")
 sys.stdout.write(f"Destination folder '{folder_name}'
↪ ID: {folder_id}\n\n")

 persisted_count = 0
 for target in targets:
     src_name = os.path.basename(target)
     dest_name = src_name

     matched_artifact_path = None
     if src_name in artifact_map:
         matched_artifact_path = src_name
     else:
         for art_path in artifact_map:
             if os.path.basename(art_path) == src_name:
                 matched_artifact_path = art_path
                 break
```

```
        if not matched_artifact_path and src_name.endswith(".
↪ pdf"):
            for art_path in artifact_map:
                if art_path.endswith(".pdf"):
                    matched_artifact_path = art_path
                    break

        if not matched_artifact_path:
            sys.stdout.write(f"⚠️  Artifact '{src_name}'
↪ was not produced by this build run (skipping).\n")
            continue

        url = artifact_map[matched_artifact_path]
        content = download_build_artifact(session, url,
↪ routing)

        if content is None:
            sys.stderr.write(f"❌ Failed to download
↪ artifact data for '{matched_artifact_path}'.\n")
            continue

        sys.stdout.write(f"📦 Fetched '{
↪ matched_artifact_path}' ({len(content)} bytes) from
↪ build container.\n")
        success = upload_to_folder(
            session=session,
            project_id=project_id,
            csrf_token=csrf_token,
            file_bytes=content,
            dest_name=dest_name,
            folder_id=folder_id,
        )

        if success:
            sys.stdout.write(f"✅ Successfully persisted '{
↪ folder_name}/{dest_name}' into the project tree.\n")
            persisted_count += 1
```

```
    sys.stdout.write(f"\nDone. {persisted_count}/{len(
    ↪ targets)} files successfully persisted to Overleaf
    ↪ project.\n")


if __name__ == "__main__":
    main()
```

Listing 39: Compile and upload the output.pdf into a folder in the Overleaf project. Note that LaTeX commands were added for layout purposes.

```
./copy_files_from_build_to_project.py
    ↪ README_programmer_notes_Biber_and_BibLaTeX.tex output.
    ↪ pdf
Initiating compilation on Overleaf (target: README_notes/
    ↪ README_programmer_notes_Biber_and_BibLaTeX.tex)...
Compilation Status: success
Total build artifacts returned: 177
Destination folder 'artifacts_1791413684' ID:
    ↪ 6ac6cdb4a98c95d1223e6ce3

📦 Fetched 'output.pdf' (311800 bytes) from build container
    ↪ .
✅ Successfully persisted 'artifacts_1791413684/output.pdf'
    ↪  into the project tree.

Done. 1/1 files successfully persisted to Overleaf project.
```

4.3 Risks of the Method 1 based automation approach

The automation scripts introduce several technical, architectural, and operational vulnerabilities that are important to consider:

1. Fragility of Undocumented Internal APIs

   The scripts rely entirely on Overleaf's private, undocumented frontend endpoints (`/updates`, `/entities`, `/compile`, `/upload`) rather than a public, supported API. Because these endpoints power Overleaf's internal web application UI rather than external developer integrations, **any minor frontend update by Overleaf can instantly break the scripts**. Changes

to JSON schemas, URL routing, or header requirements will cause silent failures or `HTTP 400/422` errors without warning.

2. Local Cookie Scrape Dependency (`browser_cookie3`)

   The authentication mechanism depends on reading live browser cookies (`overleaf_session2`) from the local machine using libraries like `browser_cookie3`. This introduces severe environment-specific constraints:

   - **Operating System Keychains**: Modern operating systems encrypt browser cookie databases using OS-level keyrings (macOS Keychain, Linux GNOME Keyring/KWallet), which frequently block automated Python scripts from reading them without explicit user intervention.
   - **CI/CD Incompatibility**: This method **cannot** be run inside standard headless CI/CD runners (like GitHub Actions) because there is no local browser profile or active session cookie available in a clean cloud container.

3. Hardcoded Fallbacks and Security Risks

   If a user runs the script without explicitly setting the `OVERLEAF_PROJECT_ID` environment variable, the script defaults to interacting with a project that is statically configured in the code. While it requires authentication cookies, hardcoding project IDs or server strings (such as `clsi-cache-zone-b-prod-3`) introduces maintenance debt and a risk of cross-project interference.

4. High Friction and Steep Learning Curve for Users

   For a typical doctoral student who just wants to write a thesis, maintaining a Python script that reverse-engineers a SaaS platform's session management creates an enormous cognitive and technical barrier. If a student changes their default browser, updates their operating system, or clears their cookies, the automation tool breaks, shifting troubleshooting overhead onto the user.

5. The Maintenance Loop of Reverse Engineering

   As seen in the script's code, handling edge cases requires complex fallback parsing (such as `regex` scans over raw text payloads, handling `FineUploader` parameters for file uploads, and timestamping fallback

folders like `base_folder_name_int(time.time())`). Maintaining this wrapper code requires continuous developer attention whenever Overleaf modifies its backend infrastructure.

The primary vulnerability of this approach is that **it couples the thesis workflow to a closed third-party platform's private implementation details**. While it serves as a tactical workaround for Overleaf's limitations, its long-term viability depends on how long Overleaf's current internal routing remains unchanged. This underscores why Method 2 (moving the source of truth to a GitHub repository and compiling natively via GitHub Actions) shown in Section 4.1 remains the more robust, future-proof architectural choice for continuous integration.

List of Acronyms and Abbreviations

API	Application Programming Interface
CI	Continuous Integration
CLSI	Common LaTeX Service Infrastructure
CReDiT	Contribution Roles Taxonomy
DiVA	Digitala Vetenskapliga Arkivet
DOI	Digital Object Identifier
GUI	Geraphical User Interface
IQR	interquartile range
ISBN	International Standard Book Number
ISNI	International Standard Name Identifier
ISO	International Standard Organization
ISSN	International Standard Serial Number
JSON	JavaScript Object Notation
KTH	KTH Royal Institute of Technology
MODS	Metadata Object Description Schema
ORCID	Open Researcher and Contributor ID
PDF	Portable Document Format
RFC	Request for Comments
SaaS	Software as a Service
UI	User Interface
URL	Uniform Resource Locator
XML	Extensible Markup Language

References

[1] **[Unresolved Placeholder: Placeholder fake reference for use in examples]**. Gerald Q. Maguire Jr., Jane Smith, and Alex Green, "Collaborative systems in higher education," *Journal of Academic Stewardship* vol. 3(2), pp. 112–128, 2025, ISSN: 0000-0000. DOI: 10.1000/182

[2] National Information Standards Organization, *Contributor Roles Taxonomy, ANSI/NISO Z39.104-2022*, ANSI/NISO Z39.104-2022, Standard, National Information Standards Organization, Baltimore, MD, USA: NISO, Feb. 8, 2022, ISBN: 978-1-950980-18-5. DOI: 10.3789/ansi.niso.z39.104-2022 [Online]. Available: https://www.niso.org/standards-committees/credit Accessed: Aug. 31, 2026.

[3] **[Unresolved Placeholder: Placeholder fake reference for use in examples]**. Gerald Q. Maguire Jr. and Jane Smith, "Automated metadata integrity in academic publishing," *Journal of Academic Stewardship* vol. 4(1), pp. 45–60, 2026, ISSN: 0000-0000. DOI: 10.1000/182

[4] International Organization for Standardization, *Information and documentation, International standard name identifier (ISNI)*, 2nd ed., ISO 27729:2024, Geneva: International Organization for Standardization, Nov. 2024, 12 pp. [Online]. Available: https://www.iso.org/standard/87177.html

[5] International Organization for Standardization, *Information and documentation, International standard book number (ISBN)*, 5th ed., ISO 2108:2017, Geneva: International Organization for Standardization, Dec. 2017, 22 pp. [Online]. Available: https://www.iso.org/standard/65483.html

[6] D. Eastlake 3rd and A. Panitz, *Reserved Top Level DNS Names*, RFC 2606, Jun. 1999. DOI: 10.17487/RFC2606 [Online]. Available: https://www.rfc-editor.org/info/rfc2606/

[7] S. Cheshire and M. Krochmal, *Special-Use Domain Names*, RFC 6761, Feb. 2013. DOI: 10.17487/RFC6761 [Online]. Available: https://www.rfc-editor.org/info/rfc6761/